PACE

Personal Fitness & Nutrition Platform

Final Product & Engineering Specification — v2.0

Reviewed and consolidated implementation brief for the coding agent: scope decisions, architecture, formulas with test vectors, injury-safety model, database schema and RLS, seed data, phased plan, acceptance criteria and a full edge-case catalogue.

Audience: the coding agent and any developer building PACE. This document is the single source of truth; where it conflicts with the original PRD (v1), this document wins.

Platforms	iOS and Android
Stack	React Native · Expo · TypeScript (strict) · Expo Router · Supabase
Status	Source of truth — implement phase by phase
Date	8 November 2026

Contents

0. How to use this document . . . 7

1. Review summary — what changed and why . . . 8

1.1 Kept as-is (core value) . . . 8

1.2 Removed or deferred (unnecessary for this kind of app, or not workable yet) . . . 8

1.3 Ambiguities resolved (one choice each) . . . 9

1.4 Added (missing from v1 but needed for a trustworthy coach) . . . 9

2. Product scope and release plan . . . 11

2.1 Vision . . . 11

2.2 Release scope . . . 11

2.3 Out of scope (explicitly) . . . 11

3. Non-negotiable rules . . . 12

3.1 Safety . . . 12

3.2 Data integrity . . . 12

3.3 Honesty . . . 12

3.4 Social (from V1.2, Section 33) . . . 12

4. Technology stack (decided) . . . 13

5. Architecture . . . 14

5.1 Layering rules . . . 14

5.2 Folder structure . . . 14

5.3 Error handling . . . 14

6. Navigation and screens . . . 15

6.1 Root routing logic . . . 15

6.2 Tabs . . . 15

6.3 Quick actions . . . 15

6.4 V1 screen inventory . . . 15

6.5 Optional features (Settings → App features) . . . 15

7. Onboarding . . . 17

8. Calculation engine (normative) . . . 18

8.1 Age . . . 18

8.2 BMR (Mifflin-St Jeor) . . . 18

8.3 TDEE . . . 18

8.4 Calorie target . . . 18

8.5 Safety floor . . . 18

8.6 Macronutrients . . . 18

8.7 Water . . . 19

8.8 Projection . . . 19

8.9 Weight trend and recalibration . . . 19

8.10 Exercise calories (estimate) . . . 19

8.11 Training load . . . 19

8.12 Unit conversion constants . . . 20

9. Injury and safety model . . . 21
9.1 Status stages (user-set) . . . 21
9.2 Body area → restriction tags . . . 21
9.3 Pain rule . . . 21
9.4 Disclaimer text (onboarding, health screen, legal) . . . 21
10. Training module . . . 22
10.1 Concepts . . . 22
10.2 Plan generator (domain/training/scheduler.ts) . . . 22
10.3 Calendar . . . 22
10.4 Workout execution . . . 22
10.5 Progression for light equipment (domain/training/progression.ts) . . . 23
10.6 Exercise library . . . 23
11. GPS activity tracking . . . 24
11.1 Flow . . . 24
11.2 Permissions . . . 24
11.3 Sampling settings . . . 24
11.4 Filtering (domain/activity/gps-filter.ts) . . . 24
11.5 Metrics . . . 24
11.6 Resilience . . . 24
11.7 Route storage and privacy . . . 25
11.8 Manual activity . . . 25
12. Nutrition module . . . 26
12.1 Data sources . . . 26
12.2 Logging . . . 26
12.3 Recipes (TheMealDB) . . . 26
12.4 Measure parser and scaling (domain/nutrition) . . . 26
12.5 User recipes . . . 27
12.6 Meal planner (V1 simple) . . . 27
12.7 Shopping list . . . 27
12.8 Water . . . 27
13. Weight and body progress . . . 28
13.1 Weight logging . . . 28
13.2 Chart . . . 28
13.3 Statistics . . . 28
13.4 Goal reached . . . 28
13.5 Body measurements . . . 28
13.6 Progress photos (V1.1) . . . 28
14. Milestones and personal records . . . 30
14.1 Principles . . . 30
14.2 Catalogue . . . 30
14.3 Personal records . . . 30

15. Daily coach (rule-based recommendations) . . . 31
15.1 Engine . . . 31
15.2 Rule set V1 . . . 31
16. Reports . . . 32
17. Notifications (local, V1) . . . 33
18. Offline and synchronisation . . . 34
18.1 Scope . . . 34
18.2 Mechanism . . . 34
19. Security and privacy . . . 35
20. Database schema (V1) . . . 36
20.1 Identity and preferences . . . 36
20.2 Health and check-ins . . . 36
20.3 Body tracking . . . 37
20.4 Training . . . 37
20.5 Activities . . . 38
20.6 Nutrition . . . 38
20.7 Progress and reports . . . 39
20.8 Migrations . . . 40
21. Row Level Security and Storage policies . . . 41
21.1 Standard user-owned table . . . 41
21.2 Shared catalogue tables (exercises, workout_templates, foods, recipes) . . . 41
21.3 Storage . . . 41
21.4 RLS tests (pgTAP, required) . . . 41
22. Integrations . . . 42
22.1 Provider interfaces . . . 42
22.2 TheMealDB . . . 42
22.3 Open Food Facts (V1.1) . . . 42
22.4 Failure policy . . . 42
23. Design system . . . 43
23.1 Tokens . . . 43
23.2 Components (src/components/ui) . . . 43
23.3 Accessibility . . . 44
23.4 Copy tone . . . 44
23.5 Touch feedback, transitions and haptics . . . 44
24. Environment and configuration . . . 46
24.1 Variables . . . 46
24.2 Environments . . . 46
25. Seed and reference data . . . 47
25.1 Example user (development seed only) . . . 47
25.2 Exercise library seed (minimum) . . . 47
25.3 Workout templates (seed) . . . 49

25.4 MET reference (from the Adult Compendium of Physical Activities; verify values during implementation) . . . 50
25.5 Milestone catalogue . . . 50
25.6 Unit and display reference . . . 51
26. Implementation phases . . . 52
Phase 1 — Foundation . . . 52
Phase 2 — Dashboard and progress . . . 52
Phase 3 — Training . . . 52
Phase 4 — Nutrition . . . 52
Phase 5 — GPS activity . . . 52
Phase 6 — Personal coach . . . 52
Phase 7 — Production readiness . . . 52
Phase 8 — Social (V1.2), in four reviewed stages . . . 53
27. Acceptance criteria (V1) . . . 54
28. Edge-case catalogue . . . 55
28.1 Authentication and session . . . 55
28.2 Onboarding and profile . . . 55
28.3 Calculations . . . 55
28.4 Weight and measurements . . . 56
28.5 Training . . . 56
28.6 GPS activity . . . 56
28.7 Nutrition and recipes . . . 57
28.8 Milestones, records, reports . . . 57
28.9 Coach and notifications . . . 57
28.10 Sync and data lifecycle . . . 58
29. Test plan and test vectors . . . 59
29.1 Levels . . . 59
29.2 Calculation vectors . . . 59
29.3 Weight vectors . . . 59
29.4 Geo and activity vectors . . . 59
29.5 Nutrition vectors . . . 60
29.6 Device test matrix (minimum) . . . 60
30. Implementation rules for the coding agent . . . 61
31. Phase report template . . . 62
32. Kickoff prompt (Phase 1) . . . 63
33. Social (V1.2, Phase 8) . . . 64
33.1 Concepts and defaults . . . 64
33.2 Data model (all tables RLS-protected, pgTAP-tested) . . . 64
33.3 Access rules . . . 64
33.4 Leaderboards and challenges . . . 65
33.5 Screens . . . 65
33.6 Push notifications . . . 65

33.7 Moderation, privacy and accounts . . . 65
33.8 Testing . . . 66
Appendix A — Glossary . . . 67
Appendix B — External references . . . 68

0. How to use this document

- Read the whole document once. Then implement **one phase at a time** (Section 26). Stop after each phase and produce the Phase Report (Section 31).
- Every "MUST" is a requirement. "SHOULD" is strongly recommended. "MAY" is optional.
- Formulas, constants, rounding rules and test vectors in Sections 8, 29 are normative. Unit tests MUST reproduce the test vectors exactly.
- The edge-case catalogue (Section 28) is part of the acceptance criteria. A phase is not done until every edge case for that phase's modules is handled and, where marked, tested.
- Do not add features that are not in the current phase. Do not silently drop a requirement: if something cannot be done, list it under "Known limitations" in the phase report.
- The example user (Section 25.1) is development seed data only. Never hardcode it.

1. Review summary — what changed and why

The original PRD was strong on vision but mixed V1 and V2 scope, left several technical choices open, and contained a few features that cannot work as described with the chosen data sources. This section records every decision.

1.1 Kept as-is (core value)

- Five-tab app: Home, Training, Nutrition, Progress, Profile.
- Mifflin-St Jeor BMR, activity-multiplier TDEE, moderate deficit, configurable macros, transparent calculations, user-confirmed target changes.
- Weight tracking with 7-day rolling average, range filters and honest projections.
- Training calendar, exercise library, workout execution with sets/reps/RPE.
- GPS walking, running and cycling tracking with filtering and local buffering.
- Nutrition logging with snapshot preservation, TheMealDB recipes with no invented nutrition.
- Rule-based daily coach, milestones, personal records, weekly report.
- RLS on every user table, no service keys in the client, account deletion, data export.

1.2 Removed or deferred (unnecessary for this kind of app, or not workable yet)

Item from PRD v1	Decision	Reason
Supabase Realtime	Removed	Single-user personal data. No live collaboration. Adds cost and complexity for nothing.
Expo Sensors	Removed	Not needed. Step counts come from HealthKit/Health Connect in V2.
Edge Functions generate-plan, calculate-targets, process-milestones, generate-weekly-report, sync-recipes	Replaced	All calculation logic lives in a pure TypeScript domain module (`src/domain`). It works offline, is unit-testable, and the user may edit their own targets anyway, so there is no need for server authority. Only two Edge Functions remain: `delete-account` and `recipes-proxy`.
`activity_route_points` table	Removed	One row per GPS point is expensive and a privacy risk. Routes are stored as one private compressed file per activity in Storage.
`notifications` table	Removed	All V1 reminders are local notifications. Only preferences are stored.
`daily_targets` row per day	Replaced	Replaced by versioned `nutrition_targets` with `effective_from`. A target applies until a newer version starts. Preserves history without writing a row every day.
Denormalised totals on `meal_logs`	Removed	Totals are derived from items (view). Prevents drift between header and items.
Meal suggestions that "hit remaining macros" using TheMealDB	Replaced	TheMealDB provides no nutrition and no servings. Macro-matched suggestions only use recipes that have reliable nutrition (user recipes built from foods). TheMealDB is an inspiration source.
Pantry / "available ingredients"	Removed	High effort, low value for one user.
Budget preference, currency	Removed	No data source has cost information. Nothing would use it.
Occupation question + steps + activity level (overlapping)	Simplified	One activity-level question with clear descriptions, plus optional average steps.
Multiple goals each with priority	Simplified	One primary goal (drives calories) plus optional focus tags (strength, sport, running, mobility, cardio). The sports a user plays (any number, or none) are stored separately (`user_preferences.sports`).
Monthly reports	V1.1	Weekly report first; monthly reuses the same engine.
Barcode scanning (Open Food Facts)	V1.1	Moved earlier than "future": it is the fastest way to log packaged food. Ships right after Phase 4.

Item from PRD v1	Decision	Reason
Progress photos	V1.1	Highly sensitive data. Schema and storage policy designed now, UI later.
Elevation gain	V2	Phone GPS altitude is unreliable. Raw altitude is stored so it can be computed later.
Training-day carbohydrate cycling	V1.1 (off by default)	Nice-to-have; same daily target is simpler and adequate.
HealthKit / Health Connect, AI coach, always-on background location	V2	As in PRD.
Social features (friends, feed, clubs, challenges, leaderboards)	V1.2 (Phase 8)	Private by default; leaderboards rank effort only, never weight, body or food (Section 33).
Subscriptions	Future	As in PRD.

1.3 Ambiguities resolved (one choice each)

Open choice in v1	Decision
NativeWind or StyleSheet	Custom StyleSheet design system with theme tokens. No extra build tooling.
Jest or Vitest	Jest via `jest-expo` preset (best React Native compatibility).
Chart library	`victory-native` (Skia-based, requires `@shopify/react-native-skia`).
Maps	`react-native-maps` (Apple Maps on iOS, Google Maps on Android — needs an Android Maps API key).
Background GPS V1 or V2	V1 includes background tracking **only during a user-started activity** (otherwise tracking stops when the phone is locked, making the feature useless). Always-on / passive background location is not used at all.
Exercise calories and TDEE double counting	The activity multiplier already includes planned exercise. Logged exercise calories are shown as estimates but are **not** added to the food budget.
Where TDEE calibration happens	Domain module, every 14 days from day 21, proposals require confirmation (Section 8.9).
Minimum age	18+ in V1 (calorie deficits for minors need professional oversight; also simplifies GDPR consent).
Week start	Monday by default, configurable.

1.4 Added (missing from v1 but needed for a trustworthy coach)

- **Daily check-in** (optional): sleep hours, fatigue 1–5, soreness 1–5, pain 0–10 with body area. Required input for recovery rules.
- **Injury model** with status stages, restriction tags and exercise contraindication tags (Section 9). Makes the "recent right ankle sprain" requirement actually enforceable.
- **Workout templates** with a 20-minute short version and injury-safe alternative per exercise, so the app stays useful when time is short.
- **Progression rules for light equipment** (5 kg dumbbell/kettlebell): reps, tempo, unilateral, pauses before load.
- **Any sport, or none.** A catalogue of 17 sports (`src/domain/sports/catalogue.ts`: football, futsal, basketball, handball, volleyball, tennis, padel, badminton, table tennis, hockey, rugby, water polo, martial arts/boxing, swimming, running club, cycling club, other). Each sport has its own words for games and practices, METs, the body areas it loads and a drill pack. Users with no sport see no sport wording.
- **Game-day scheduling rules** (MD-1, MD+1) for games of any sport, and logging games/practices with duration and RPE.
- **Session training load** = RPE × minutes, used for recovery rules.
- **Recurring events** model with idempotent generation (no duplicate occurrences).
- **Reference food catalogue** (USDA FoodData Central subset, public domain) plus food portions. Without it, food search has no reliable data source in V1.
- **User recipes** built from catalogue foods, giving accurate per-serving nutrition.
- **Saved meals** for quick repeat logging.
- **"Day complete" flag** for nutrition, so calibration and reports can tell incomplete logging from low intake.

- **Consents** table (terms, privacy, health data, location, notifications) with versions.
- **Client-generated UUIDs + outbox** for offline-safe, duplicate-free sync.
- **Target adjustment audit** (`target_adjustments`): every proposed and accepted change is recorded with its inputs.
- **Manual activity entry** fallback (treadmill, GPS denied, forgot to start).
- **Undo** for quick logs (water, weight) for 5 seconds.
- **Production details**: TheMealDB key via server proxy, Open Food Facts user-agent and licence, EU data region, iOS 64 pending-notification limit.

2. Product scope and release plan

2.1 Vision

PACE is a personal fitness coach that turns a user's profile, schedule, equipment, injuries and recorded data into a realistic weekly plan and a daily set of clear, explained recommendations. It must be useful on a busy day (a 20-minute option, one-tap logging) and safe on a bad day (injury and fatigue aware).

2.2 Release scope

Release	Contents
V1 (MVP)	Auth, onboarding, plan preview, dashboard, weight + measurements, goals, milestones, PRs, exercise library, templates, calendar, recurring events, workout execution + history, nutrition logging, food catalogue, custom foods, saved meals, water, TheMealDB browser, user recipes, servings scaling, simple meal planner, shopping list, GPS walk/run/cycle (foreground + background during active session), manual activities, daily check-in, rule-based coach, target recalibration, weekly report, local notifications, offline outbox, data export, account deletion.
V1.1	Barcode scanning + Open Food Facts search, progress photos, monthly report, training-day carb adjustment, GPX export, best-split PRs (fastest 1 km / 5 km within a run).
V2	HealthKit / Health Connect, AI coach (backend adapter, read-only suggestions), elevation, privacy zones for routes, light theme polish, Apple Watch / Wear OS.
V1.2 — Social (Phase 8A–8D)	Friends and social profiles, activity feed with cheers and comments, push notifications, private clubs, challenges and effort-only leaderboards (Section 33).
Future	Subscriptions, coaching marketplace, discoverable public clubs.

2.3 Out of scope (explicitly)

Medical diagnosis, rehabilitation programmes, meal prescriptions for medical conditions, leaderboards or challenges on weight, body measurements, calories or food, public profiles, public route sharing, contact-book upload, sale or sharing of user data. (Effort-only leaderboards among friends and clubs are in scope from V1.2, Section 33.)

3. Non-negotiable rules

3.1 Safety

1. The app is not a medical device. It never diagnoses, never prescribes rehabilitation, never recommends training through pain.
2. Active or recovering injuries automatically exclude contraindicated exercises (Section 9). The app never upgrades injury status itself; only the user can, and the app recommends professional clearance before returning to sport.
3. Calorie targets never go below the safety floor (Section 8.5). Manual edits below the floor are blocked with an explanation.
4. Weekly weight-loss rate above 1.0% of body weight is not selectable. Above 0.75% requires an explicit warning confirmation.
5. Target weights implying BMI < 18.5 are not accepted for weight-loss goals.
6. Milestones never reward speed of weight loss, very low intake, or excessive training volume.
7. Calorie targets are never changed automatically. All changes are proposals the user accepts or rejects.
8. Pain ≥ 4/10 reported in a check-in converts that day's planned training for that area to rest and shows "consider professional advice".

3.2 Data integrity

1. Never silently discard user-entered data. Failed syncs remain visible and retryable.
2. Logged food stores a nutrition snapshot. Editing or deleting a food never changes history.
3. Missing data is shown as missing, never as zero.
4. All timestamps stored as `timestamptz` (UTC). Every user event also stores a `local_date` computed at the time of the event in the user's timezone. `local_date` is never recomputed later.
5. All internal measurements are metric (kg, cm, m, ml, kcal). Conversion happens only at the UI edge.
6. Every user-created row has a client-generated UUID so retries are idempotent.

3.3 Honesty

1. Estimates (TDEE, calories burned, projections, GPS distance) are labelled as estimates.
2. No invented recipe quantities or nutrition. "Nutrition unavailable" is a valid, expected state.
3. No placeholder features that appear to work. Unfinished items are hidden or labelled "Coming soon".
4. No fake success: every backend write shows real success or real failure.

3.4 Social (from V1.2, Section 33)

1. **Private by default, per-type opt-in.** Nothing is visible to anyone until the user chooses; visibility is enforced by RLS on the server, never only in the UI.
2. **Never shareable:** weight, body measurements, goal numbers, food and water, check-ins and pain, injuries and health, notes, GPS routes and start/end locations, exact start times (shared items carry date and duration only).
3. **Encourage, never shame:** no weight, body or calorie competition; no "X passed you" or other comparison notifications; leaderboards are effort-only and opt-in; daily caps discourage overtraining; users with an active injury restriction get no challenge nudges.
4. **User-generated content is moderated:** report, block, word filter, removal, owner/admin removal of members, a published contact, review within 24 h.
5. **18+ only; no contact-book upload;** people are found by handle or invite link.

4. Technology stack (decided)

Area	Choice
Runtime	React Native via Expo, latest stable SDK at project start (pin the exact version), New Architecture enabled
Language	TypeScript, `strict: true`, `noUncheckedIndexedAccess: true`, no `any` (ESLint rule `@typescript-eslint/no-explicit-any: error`)
Navigation	Expo Router (file-based, `src/app`)
Animation / gestures	react-native-reanimated, react-native-gesture-handler
Images	expo-image
Server state	@tanstack/react-query (+ persisted cache for offline reads)
Client state	zustand (UI state, active workout, active activity)
Forms / validation	react-hook-form + zod (`@hookform/resolvers`)
Local DB	expo-sqlite (outbox, GPS buffer, active session drafts)
Backend	Supabase: Postgres, Auth, Storage, Edge Functions (Deno)
Session storage	AsyncStorage for the Supabase session (SecureStore has a ~2 KB value limit on iOS); optional encrypted "LargeSecureStore" pattern
Charts	victory-native + @shopify/react-native-skia
Maps	react-native-maps
Location	expo-location + expo-task-manager
Notifications	expo-notifications (local only in V1)
IDs	expo-crypto `randomUUID()`
Icons	lucide-react-native
Dates	date-fns + date-fns-tz
Testing	jest-expo, @testing-library/react-native, Maestro (E2E), pgTAP via `supabase test db` (RLS)
Lint / format	ESLint (expo config) + Prettier; CI runs typecheck, lint, tests
Builds	EAS Build (development, preview, production profiles). Background location requires a development build; Expo Go is not sufficient.
Crash reporting (optional)	Sentry with PII scrubbing; never send weight, food, health or location data

5. Architecture

5.1 Layering rules

1. **UI** (`src/app`, `src/components`, `src/features/*/components`) renders and dispatches. No calculations, no direct Supabase or HTTP calls.
2. **Feature hooks** (`src/features/*/hooks`) combine queries/mutations and domain functions.
3. **Repositories** (`src/features/*/api` or `src/integrations/supabase`) are the only code that talks to Supabase. They return typed, zod-validated objects.
4. **Domain** (`src/domain`) is pure TypeScript: no React, no I/O, no `Date.now()` (time is passed in). 100% unit-testable.
5. **Integrations** (`src/integrations`) wrap external APIs behind interfaces with normalisers.
6. **Services** (`src/services`) handle sync, notifications, storage, location tasks.

5.2 Folder structure

```
src/
  app/                      # Expo Router routes only (thin)
    _layout.tsx
    (auth)/  welcome, sign-in, sign-up, forgot-password, reset-password, verify-email
    (onboarding)/  personal, goals, activity, equipment, schedule, nutrition, health, consent, plan
    (tabs)/  index (Home), training, nutrition, progress, profile
    training/  calendar, workout/[id], session/[id], exercises/index, exercises/[id], templates/[id]
    activity/  track, [id], manual
    nutrition/  search, food/new, meal/[id], recipes/index, recipes/[id], recipe/new, planner, shopping
    progress/  weight, measurements, milestones, records, reports/[week]
    profile/  personal, goals, training, nutrition, equipment, health, units, notifications, privacy, export, delete
    check-in.tsx
  components/  ui/ charts/ forms/ cards/ feedback/
  features/   dashboard/ onboarding/ training/ exercises/ nutrition/ recipes/
              activity-tracking/ weight/ measurements/ milestones/ recommendations/ profile/ check-in/
  domain/
    calculations/   bmr.ts tdee.ts targets.ts macros.ts projection.ts recalibration.ts units.ts met.ts
    training/       scheduler.ts injury-filter.ts progression.ts load.ts volume.ts
    activity/       geo.ts gps-filter.ts pace.ts
    nutrition/      totals.ts scaling.ts measure-parser.ts shopping.ts
    weight/         rolling-average.ts trend.ts
    milestones/     catalogue.ts evaluate.ts records.ts
    recommendations/ rules.ts engine.ts
    reports/        weekly.ts
    validation/     schemas.ts (shared zod schemas)
  integrations/
    supabase/  client.ts database.types.ts (generated)
    recipes/   recipe-provider.interface.ts themealdb.provider.ts recipe-normalizer.ts
    food/      food-provider.interface.ts usda-catalogue.ts openfoodfacts.provider.ts (V1.1)
    location/  location-provider.ts background-task.ts
  services/   sync/ (outbox.ts, sync-engine.ts) notifications/ storage/ export/
  stores/  hooks/  types/  constants/  utils/  theme/
supabase/
  migrations/  seed/  functions/delete-account/  functions/recipes-proxy/  tests/ (pgTAP)
tests/  unit/ integration/ e2e/ (Maestro flows)
docs/
```

5.3 Error handling

- One `AppError` type: `{ code, message, cause?, retryable }`. Repositories map Supabase/HTTP errors into it.
- UI shows a friendly message plus Retry where `retryable`. Never show raw SQL or stack traces.
- Every screen has four states: loading (skeleton), empty (with a call to action), error (with retry), success.

6. Navigation and screens

6.1 Root routing logic

1 No session → `(auth)/welcome`.

2 Session but email not verified → `verify-email`.

3 Verified but `profiles.onboarding_completed_at` is null → resume onboarding at the last saved step.

4 Otherwise → `(tabs)`.

6.2 Tabs

Tab	Primary content
Home	Greeting and date (tapping opens Profile), coach cards, **Quick log** (two actions with tinted icon circles), **Your progress** (Start · Now · Goal with the progress bar and "2.7 kg down · 15.3 kg to go"), **Daily check-in** prompt, today's plan card (with the next fixed event as one "Next: …" line), today's nutrition bars when nutrition is on
Training	"+ Add" button, today, this week (prev/next), then only three links: History (workouts, games and activities together, with the month calendar), Personal records, Exercise library
Nutrition	Day selector, remaining kcal, macro bars, meal sections (breakfast, lunch, dinner, snacks), water, "Mark day complete", recipes, planner, shopping list
Progress	Weight chart + stats, Log weight, then Weight history, Body measurements, Milestones, Weekly report
Profile	You: personal info, goal and targets (or Goal when nutrition is off), health and injuries. Preferences: Training setup (sports, schedule, equipment), App features, Reminders, Appearance. Account: Data and privacy (export, delete, legal documents), Sync status (only while changes are waiting or failed), Sign out

One main action per screen. Rarely used screens sit one level deeper, never on the tab itself: the 2-week target review is opened from its coach card on Home (W2) or from Goal and targets; food preferences from App features; the four legal documents from Data and privacy. Screens that were removed from a tab keep their routes so notifications and deep links still work.

6.3 Quick actions

Home has two: **Log weight** and **Check-in**. Everything you can add on the Training side (record a walk/run/ride; log a gym or home workout, a game or practice, or a walk/run/ride without GPS; plan a workout) is behind the Training tab's **+ Add** button (`training/add`). Today's planned workout is started from its Today card. Logging must take ≤ 3 taps for repeat actions.

6.4 V1 screen inventory

Auth: Welcome, Sign in (with "Forgot your password?"), Sign up, Forgot password, Reset password (deep link), Verify email. Profile → Account → **Change password** (current password re-checked, then the new one; "Forgot your current password?" sends a reset link). Onboarding: About you (incl. activity level), Goals, Your week (incl. equipment), Food tracking, Health, Consent, Plan preview. Training: Add (record/log/plan), Week/month calendar, Workout detail, Active workout, Workout summary, Exercise library, Exercise detail, Custom exercise form, Template detail, History (workouts and activities combined). Activity: Activity type picker, Live tracking, Activity summary, Manual activity form, Activity detail. Nutrition: Daily dashboard, Food search, Custom food form, Meal detail, Saved meals, Recipe browser, Recipe detail (with servings), User recipe builder, Meal planner (week), Shopping list. Progress: Weight, Measurements, Milestones, Personal records, Weekly report. Long lists (weight history, training history, activities, exercises, sync queue) load **20 rows at a time**: the next page loads when scrolling near the end, and a "Showing 20 of 134 · Show more" footer does the same on a tap (`ListScreen`). Profile: all settings screens listed in 6.2; Training setup links to Sports, Schedule and Equipment, each with its own Save. Other: Daily check-in, Sync status, Legal (privacy policy, terms, disclaimer, data sources and licences).

6.5 Optional features (Settings → App features)

Nutrition tracking can be switched off (`user_preferences.nutrition_enabled`; the column defaults to on for older app versions, but onboarding step 4 asks and pre-selects **off**, so new users start training-first). When on, App features links to the food preferences (diet, allergies, cuisines, cooking time). When off:

- the Nutrition tab, Home's "Today's nutrition" card, Profile's nutrition settings and the weekly report's nutrition section are hidden; "Goal and targets" becomes "Goal" (goal and target weight);
- coach rules S4, N1–N3 and W2 are not shown (15.2); the two-week calorie review (8.9) does not run;
- meal and water reminders are not scheduled (17); the reminder settings hide them;
- links or old notifications that open a nutrition screen show "Nutrition is turned off" with a button to turn it on.

Nothing is deleted. Calorie targets are still calculated and saved, so turning nutrition back on shows exactly the same targets. Weight tracking, the goal and training are unaffected.

7. Onboarding

Rules: progress bar, back navigation, draft saved locally after each step (resume after app kill), optional fields clearly marked, "Skip" allowed on optional steps with defaults shown. All inputs validated with zod schemas shared with the domain layer.

Step	Fields	Validation / notes
1 About you	display name (req.), date of birth (req.), sex for metabolic calculation (male/female, req., explained), height (req.), current weight (req.), unit system (metric/imperial), timezone (auto-detected, editable); **activity level** (5 options, each with plain-language description that includes exercise), optional average daily steps	Name 1–50 chars. Age 18–100. Height 120–230 cm. Weight 30–350 kg. Imperial: ft 3–7, in 0–11.9, lb 66–770. Activity: show multiplier and explain it already includes exercise. Saved to the `personal` and `activity` draft keys.
2 Goals	primary goal: lose weight / maintain / build muscle / general fitness; target weight (req. for lose/build); desired weekly rate; optional target date; focus tags (strength, sport, running, cardio, mobility); sports played (multi-select from the catalogue, max 8, or "I don't play a sport")	Lose: target < current, target BMI ≥ 18.5. Rate default 0.5%/week; range 0.25–1.0%; > 0.75% needs warning confirmation. Build: target > current; rate 0.1–0.25%/week. Optional secondary target weight (e.g. 87 kg) creates an extra milestone only.
3 Your week	training days/week (0–7), preferred days, max session duration (15–120 min, default 45), preferred time, recurring sport commitments (title, type match/practice, weekday, time, duration), rest-day preferences; **equipment**: environment (home/gym/outdoors/mixed), equipment multi-select, per-item max weight (e.g. dumbbells 5 kg), custom entries	Commitments become `recurring_events`. Training days + commitments may overlap; scheduler resolves. "No equipment" is exclusive. Max weight 0.5–200 kg. Saved to the `schedule` and `equipment` draft keys.
4 Food tracking	"Track food too?" (optional, default **no**: training-first; no → the rest of the step is hidden and nutrition starts switched off, 6.5; yes → the questions below); diet type (omnivore/vegetarian/vegan/pescatarian/other), allergies, intolerances, disliked foods, preferred cuisines, meals per day (2–6, default 3 + snacks), cooking time preference (≤15/≤30/≤60/any), household servings (1–8, default 1)	Allergies are free-text chips with suggestions (peanut, tree nuts, milk, egg, fish, shellfish, soy, wheat/gluten, sesame).
5 Health (optional)	current injuries (body area, side, status, onset date, pain 0–10, notes, professional clearance), previous injuries, conditions, professional restrictions	Shown only after health-data consent. Disclaimer visible. Can skip.
6 Consent	terms + privacy (required), health data processing (required only if health info entered), notifications (optional). Location consent is asked later, at first activity.	Store in `consents` with document version.
7 Plan preview	BMR, TDEE, calorie target, macros, water, weekly schedule, rest days, milestones, projection range, calculation breakdown ("How we calculated this")	User can edit targets before confirming. Confirm writes profile completion, goal, `nutrition_targets` v1, `training_plans`, recurring events, first 8 weeks of scheduled workouts.

8. Calculation engine (normative)

All functions live in `src/domain/calculations`, are pure, and take explicit inputs (including `asOf` date). Store `calculation_version` (current `"2.1.0"`; 2.1.0 added the frequent-training deficit cap and the sport carbohydrate and hydration guidance, and gives identical results when no training is known) and the full inputs with every target version.

8.1 Age

`age = full years between date_of_birth and asOf` (birthday not yet reached this year → subtract 1). Feb 29 birthdays: treat Mar 1 as the birthday in non-leap years.

8.2 BMR (Mifflin-St Jeor)

```
male:   BMR = 10 × weight_kg + 6.25 × height_cm − 5 × age + 5
female: BMR = 10 × weight_kg + 6.25 × height_cm − 5 × age − 161
```

The equation is selected by "sex for metabolic calculation" (user-selectable, explained in onboarding). Keep full precision internally; round only for display.

8.3 TDEE

`TDEE = BMR × multiplier`

Level	Multiplier	Description shown to user
sedentary	1.2	Desk job, little or no exercise
light	1.375	Light exercise 1–3 days/week or on feet part of the day
moderate	1.55	Moderate exercise or sport 3–5 days/week
very	1.725	Hard exercise 6–7 days/week or physical job + training
extra	1.9	Athlete training twice daily or very physical job + daily hard training

Shown with the note: "This is an estimate. It includes your usual exercise, so exercise calories are not added on top. PACE refines it from your real weight trend after 3 weeks."

8.4 Calorie target

Constants: `KCAL_PER_KG = 7700`, `MAX_DEFICIT_KCAL = 750`, `MAX_DEFICIT_FRACTION = 0.25`, `MAX_SURPLUS_KCAL = 300`, `FREQUENT_TRAINING = { hardSessionsPerWeek: 4, maxDeficitKcal: 500 }`.

```
lose:
  desired_deficit = weekly_rate_pct/100 × weight_kg × 7700 / 7
  max_kcal = 500 if hard_sessions_per_week ≥ 4 else 750   # games + hard practices (2.1.0)
  deficit = min(desired_deficit, max_kcal, 0.25 × TDEE)
  raw = TDEE − deficit
maintain / general fitness:
  raw = TDEE
build muscle:
  surplus = min(weekly_rate_pct/100 × weight_kg × 7700 / 7, 300)
  raw = TDEE + surplus
target = max(round_to_10(raw), floor)
```

`round_to_10(x) = Math.round(x / 10) × 10`. If the floor raised the target, set `limited_by_floor = true` and show "Your target is limited by the safety minimum." Recompute and show the resulting projected weekly rate: `rate_kg_week = (TDEE − target) × 7 / 7700`. When the 500 kcal cap applied (`frequent_training_cap`), explain: "You train hard N times a week, so the daily deficit is limited to 500 kcal…" (lower risk of low energy availability, RED-S). Hard sessions per week come from the user's recurring games and practices with intensity hard.

8.5 Safety floor

`floor = max(sex_floor, round_to_10(BMR))`, where `sex_floor` = 1500 kcal (male equation) or 1200 kcal (female equation). Floors are constants in `src/constants/safety.ts` (configurable for development, not by end users in V1).

8.6 Macronutrients

```
reference_weight =
  target_weight_kg            if goal = lose and target set
  current weight              otherwise, unless BMI ≥ 30
  25 × height_m²              if BMI ≥ 30 and no target
protein_g = round(protein_g_per_kg × reference_weight)    default 1.8, allowed 1.6–2.2
fat_g     = round(fat_pct × target / 9)                    default 0.25, allowed 0.20–0.35
carbs_g   = floor((target − protein_g×4 − fat_g×9) / 4)
```

Validation: `|protein×4 + carbs×4 + fat×9 − target| ≤ 10`. If carbs < 50 g, warn and suggest lower fat %. **Sport carbohydrate note (2.1.0, information only):** when the user plays a sport and has ≥ 3 sessions a week (training days + games + practices) and `carbs_g < 3 × weight_kg`, show "Sport guidance suggests about X g of carbohydrate or more on training days…". The macro split is not changed. Editing rules: when the user edits macros directly, calories are derived from macros (single source of truth); when the user edits calories, carbs absorb the change. Fibre target (display only): 14 g per 1000 kcal.

8.7 Water

`water_ml = clamp(round_to_50(35 × weight_kg), 1500, 4000)`. Displayed as a general guideline, editable. Training days are not adjusted in V1; the targets screens show "On training days, drink roughly 500 ml extra per hour of training, more in heat" (`HYDRATION_ML_PER_TRAINING_HOUR`).

8.8 Projection

```
expected_rate = (TDEE − target) × 7 / 7700            (kg/week)
weeks_mid  = (current − target_weight) / expected_rate
weeks_fast = (current − target_weight) / (expected_rate × 1.25)
weeks_slow = (current − target_weight) / (expected_rate × 0.75)
```

Display as a date range rounded to months ("around July–October 2027") plus the statement "An estimate, not a promise. Weight changes are not linear." Chart shows a shaded band, never a single precise line. After 21 days with enough data, the band uses the observed trend (8.9) instead of the expected rate.

8.9 Weight trend and recalibration

Daily value = mean of that local day's weight entries. Trend slope = ordinary least-squares linear regression of daily values over the last 21 days, expressed in kg/week. Requires ≥ 10 daily values spanning ≥ 14 days, else "Not enough data".

Recalibration check runs when the app opens if ≥ 14 days since the last check and ≥ 21 days since the current target started:

1. Adherence = complete nutrition days (marked complete) / days in the window. If < 80%: no change; show "Log a few more full days so PACE can calibrate your target." No shaming language.
2. `observed_TDEE = mean_intake_complete_days − slope_kg_per_day × 7700`.
3. `actual_rate` = −slope (kg/week, positive = loss). Compare with target rate:
   - Loss > 1.0% body weight/week → propose +150 kcal.
   - Loss < 50% of target rate for this check **and** the previous check → propose −100 kcal (never below floor).
   - Otherwise → "On track — keep your current target."
4. Also consider recovery: if the average fatigue in check-ins ≥ 4/5 over the window, do not propose a decrease; suggest keeping the target and prioritising sleep.
5. Write a `target_adjustments` row (status `proposed`) with all inputs. On accept, create a new `nutrition_targets` version effective tomorrow. On reject, record and wait for the next check. Max one change per 14 days.

8.10 Exercise calories (estimate)

`kcal = MET × weight_kg × hours` (hours = moving time for GPS activities, active time for workouts). MET table in Section 25.4. Always displayed as "≈ 420 kcal (estimate)". Not added to the food budget.

8.11 Training load

`session_load = RPE (0–10) × duration_minutes`. Weekly load = sum over local week. "Load spike" = last 7 days > 1.3 × mean weekly load of the prior 3 weeks (requires 4 weeks of data). Used only for soft recommendations.

8.12 Unit conversion constants

1 lb = 0.45359237 kg · 1 in = 2.54 cm · 1 mi = 1609.344 m · 1 fl oz (US) = 29.5735 ml. Storage precision: weight 2 decimals (kg), lengths 1 decimal (cm), distance integer metres. Display precision: weight 1 decimal, distance 2 decimals km/mi, pace mm:ss. Round-trip rule: entering 6 ft 2 in must display 6 ft 2 in again.

9. Injury and safety model

9.1 Status stages (user-set)

Status	Meaning (shown to user)	Effect
active	Painful now; walking or daily activities affected	Exclude all exercises with the area's restriction tags; no sport; recommend rest and professional assessment if severe, not improving or swelling persists
recovering	Walking is comfortable; not yet back to sport	Exclude impact, jumping, lateral, sprint for that area; allow upper body, core, non-weight-bearing work if pain-free; events of a sport that loads that area show a caution banner (e.g. an ankle blocks basketball, not swimming)
cleared_training	A professional (or the user's judgement) allows gym-type training	Allow controlled strength incl. that area; still exclude jumping, lateral and sprint
cleared_sport	Cleared for sport	No restriction; suggest thorough warm-up
resolved	History only	No effect; kept as previous injury

Rules: the app never changes status itself. Every 7 days while active/recovering, a check-in card asks "How is your ankle?" and offers to update the status. Moving to `cleared_sport` shows "We recommend clearance from a healthcare professional before returning to matches." The user can still log matches they play; the app does not block logging but does not recommend sport while restricted.

9.2 Body area → restriction tags

Body area	Restriction tags
ankle / foot	impact, jumping, lateral, sprint, ankle_load_high, single_leg_balance
calf / achilles	impact, jumping, sprint, ankle_load_high
knee	impact, jumping, lateral, sprint, deep_knee_flexion
hip / groin	lateral, sprint, deep_hip_flexion
hamstring	sprint, hinge_loaded
lower back	spinal_loaded, hinge_loaded, impact
shoulder	overhead, push_loaded
elbow / wrist / hand	wrist_load, grip_heavy
neck	overhead, spinal_loaded
other	none automatically; user selects tags manually

Stage filtering: `active` and `recovering` exclude every tag listed for the area; in addition, `active` replaces any session that loads that area (e.g. lower-body strength for an ankle injury) with an upper-body/core or recovery session. `cleared_training` excludes only impact, jumping, lateral and sprint for the area. `cleared_sport` and `resolved` exclude nothing. Each exercise in the library has `contraindication_tags`; exercise is excluded if the intersection is non-empty. Each template exercise has an `alternative_exercise_id` used when excluded; if the alternative is also excluded, the exercise is dropped and the workout shows "Adjusted for your ankle injury".

9.3 Pain rule

Check-in pain ≥ 4/10 (any area) → today's sessions loading that area become "Rest suggested". During an active workout, the "Pain" button ends the exercise, logs it, and suggests stopping. Never "push through".

9.4 Disclaimer text (onboarding, health screen, legal)

"PACE is not a medical device and does not diagnose or treat injuries or medical conditions. Recommendations are general fitness guidance. Stop exercising if you feel pain, dizziness or chest discomfort, and consult a qualified healthcare professional about injuries, medical conditions or before starting a new programme."

10. Training module

10.1 Concepts

- **Exercise**: library item (system or user-owned).
- **Workout template**: ordered list of exercises with sets/reps/rest, intensity, estimated minutes, short version, alternatives.
- **Recurring event**: weekly repeating commitment: a game or practice of one of the user's sports (`<sport>_match` / `<sport>_practice`, e.g. `basketball_match`), a run, walk, ride, workout or other fixed activity. Added in onboarding or later under Settings → Your sports.
- **Scheduled workout**: one dated occurrence (from plan, recurrence, or user). Holds a snapshot of the template's exercises at scheduling time.
- **Workout session**: what actually happened (started/ended, sets, RPE, feeling). Can exist without a scheduled workout.

10.2 Plan generator (domain/training/scheduler.ts)

Inputs: available days, max duration, recurring commitments, equipment, focus tags, injuries, week start. Output: 7-day pattern applied to the next 8 weeks.

Hard constraints:

1 Commitments (matches/practices) are fixed and count as **hard** days.

2 MD-1 (day before a match): rest or mobility only. No hard or lower-body strength.

3 MD+1 (day after a match): recovery only (easy walk, mobility).

4 Never more than 2 consecutive hard days.

5 At least 1 full rest day per week.

6 Hard sessions per week (including matches) ≤ 4.

7 Sessions never exceed max duration; if max < 30 min, use short versions.

8 Exercises filtered by equipment and injury rules.

Soft preferences: preferred days first; alternate upper/lower or full-body; spread strength sessions ≥ 48 h apart; put lower-body strength at MD-2 or earlier. The weekly **sport-skills** session (first sport's pack; needs a sport, the sport focus, ≥ 3 training days and no restriction for that sport) avoids MD-2 and days next to a hard commitment when another training day is free, never displaces lower-body strength, and counts towards the hard-day warning.

Generation is idempotent: occurrences are keyed by `(recurring_event_id, scheduled_date)` or `(training_plan_id, scheduled_date, slot)` with unique constraints. Regeneration after settings changes only replaces future `planned` occurrences; completed, skipped and user-moved ones are untouched.

10.3 Calendar

Week view (default) and month view. Users can: view, drag or "Move to…" a workout (warn when moving onto MD-1 or adding a third consecutive hard day), mark rest day, add a one-off workout or match, add/edit recurring events ("This occurrence" vs "This and future"), edit duration, skip. Completed vs planned shown with distinct icons and colours. Rolling horizon: on app open ensure scheduled occurrences exist for the next 8 weeks.

10.4 Workout execution

- Shows current exercise, instructions, target sets × reps (or time), suggested weight, rest timer.
- Per set: record reps, weight (hidden for bodyweight), duration (timed exercises), completed checkbox. Pre-filled with last session's values.
- Actions: skip exercise, swap to alternative, add note, pause/resume, "It hurts" (pain) button, **finish at any time** (header "Finish"; if exercises are unfinished, a confirmation says how many and that unfinished sets are saved as not done).
- **Add an exercise the plan didn't include** (e.g. push-ups): search the library (system and custom; exercises that clash with an injury say "not for your current injury"), inserted after the current exercise and marked "Added by you". Sets are

pre-filled from the last time it was done, else its defaults. Added exercises have `targetSets = 0`, so extra work never lowers the completion rate; they can be removed. Any set can be added (max 10) or removed while not done (an exercise keeps ≥ 1 set).

- **Timed sets** have a "Start" countdown for the hold; when it ends the set is marked done (haptic + "Hold done" announcement).
- The workout title is shown in the header; a planned workout that is running shows "Resume workout" instead of "Start workout".
- Rest timer uses absolute timestamps (survives backgrounding) and schedules a local notification for rest end.
- Draft is persisted to SQLite after every set; reopening the app offers "Resume workout".
- Finish: total duration (excluding pauses), exercises completed, sets, volume (Σ reps × weight for weighted sets), RPE 0–10, feeling 1–5, notes. Status = completed if ≥ 1 set completed, else "abandoned" (not counted for milestones).
- **Duration to save**: Timer (default) · Planned (the planned minutes) · Enter time (date, start time, minutes; saved with `source = 'manual'`). When the timer ran under 5 minutes or over 4 hours, a hint suggests entering the real time. Calories are always estimated from the duration that is saved.
- **Log a past workout** (Training → "Log a past workout", or "I already did this" on a planned workout dated today or earlier): choose a template or "Something else" (with a name), a day (today / yesterday / earlier), start time and minutes, then the exercises: planned sets start ticked with their planned values (untick what you skipped), exercises you add start ticked, and sets can be edited, added or removed. Saved with `source = 'manual'`; from a planned workout it completes that occurrence.
- **Edit a saved workout** (history → "Edit workout"): name, date, start time, minutes, every set (reps, weight, seconds, done), add or remove sets and exercises, RPE, feeling and notes. The session keeps its id and its link to the planned workout (without completing it again); old sets are deleted and the new ones written through the outbox. A completed planned workout links to what was done ("See what you did").
- **Time rules for logged and edited workouts** (`manualTiming`): 1–300 minutes; up to 60 days back; a workout cannot start or end more than 5 minutes in the future.
- History shows one card per exercise **as it was done**: the same exercise in the warm-up and the main part stays two cards (a new group starts when the exercise changes or the set number restarts).
- Sport events (match/practice) are logged via a quick form: duration, RPE, notes. They count for training load and minutes.

10.5 Progression for light equipment (domain/training/progression.ts)

Double progression: each exercise has a rep range (e.g. 10–15). When all working sets reach the top of the range at RPE ≤ 8, next time suggest, in order: (1) +1–2 reps above range up to 20, (2) slower tempo 3-1-1, (3) unilateral variant, (4) add a set (max 4), (5) reduce rest by 15 s (min 45 s). If the user has heavier equipment, suggest the next weight step. If RPE ≥ 9.5 on two consecutive sessions, suggest repeating the same targets.

10.6 Exercise library

Fields as in schema (Section 20). Searchable by name, filterable by category, muscle group, equipment, difficulty, "safe for my injuries". Custom exercises: name, category, equipment, muscles, tracking type, instructions, caution notes. Deleting a used custom exercise archives it (history preserved). Seed list: Section 25.2.

11. GPS activity tracking

11.1 Flow

Pick type (walk/run/cycle) → permission check → "Acquiring GPS" (shows accuracy) → Start → live screen (elapsed time, moving time, distance, current speed/pace, average pace/speed, estimated kcal, GPS quality, map) → Pause/Resume → Finish (confirm) or Discard (confirm) → Summary → Save.

11.2 Permissions

- Foreground permission requested at the first activity start, with a pre-prompt explaining why.
- Background permission requested separately with a pre-prompt: "To keep recording when your screen is locked, PACE needs location access in the background **only while an activity is recording**." If denied, tracking works only while the app is open; show a persistent banner.
- iOS: `UIBackgroundModes: location`, purpose strings, `showsBackgroundLocationIndicator: true`. Android: foreground service with a notification ("PACE is recording your run"), `ACCESS_BACKGROUND_LOCATION` declared with Play Console justification. Verify the exact requirements against the current expo-location documentation at implementation time.
- Approximate/reduced accuracy: detect (accuracy > 100 m consistently) and warn that distance will be unreliable.
- Denied: offer manual activity entry.

11.3 Sampling settings

`Location.Accuracy.High` (BestForNavigation for running optional), `timeInterval` 2000 ms, `distanceInterval` 5 m, `activityType` Fitness (iOS), `pausesUpdatesAutomatically: false`. Points are written to SQLite immediately (task callback), never held only in memory.

11.4 Filtering (domain/activity/gps-filter.ts)

A point is rejected if any is true:

1. `accuracy > 25 m` (constant `MAX_ACCURACY_M`).
2. Timestamp ≤ previous accepted timestamp (duplicate/out of order).
3. Implied speed from previous accepted point > cap: walk 3.5 m/s, run 8 m/s, cycle 25 m/s.
4. Distance from previous accepted point < max(3 m, 0.5 × accuracy) → not rejected but not added to distance (jitter); timestamp still updates.

Distance = Σ haversine between consecutive accepted points **within a segment**. A pause ends a segment; no distance is added between segments. A gap > 30 s without points inside a segment is bridged only if the implied speed is under the cap. GPS quality = good (≥ 90% of points accepted, median accuracy ≤ 10 m), fair (≥ 70%, ≤ 20 m), poor (otherwise). Poor-quality activities are excluded from pace/distance PRs.

11.5 Metrics

- Haversine with R = 6 371 000 m.
- Moving time = elapsed time − paused time. Pace (walk/run) = moving_seconds / (distance_m / 1000) in s/km; speed (cycle) = distance_m / moving_seconds in m/s.
- Pace and speed are not shown until distance ≥ 100 m. Current pace uses the last 30 s of accepted points.
- kcal via MET by average speed (Section 25.4).

11.6 Resilience

- Activity UUID created at Start; the recording state (status, segments, timestamps) is in SQLite. If the OS kills the app, on relaunch show "You have an unfinished activity" → Resume (if < 6 h old) or Finish with recorded data, or Discard.

- Finish is idempotent (double-tap safe). Upload: summary row upsert + route file upload; both retried via outbox. The activity is visible locally as "Not synced yet" until done.
- Activities < 60 s or < 50 m: ask "Save this short activity?".
- The type can be changed after saving; metrics and PR eligibility are recomputed from stored points.

11.7 Route storage and privacy

Route file: `routes/{user_id}/{activity_id}.json.gz` with `{ version, segments: [[{t, lat, lng, alt, acc}]] }` using accepted points only. Bucket private. User can delete a route (keeps summary stats) or delete the whole activity. Routes are never public. Never send coordinates to analytics or logs.

11.8 Manual activity

Type (walk/run/cycle/other incl. treadmill), date/time, duration, optional distance, notes. Calories estimated if distance present; source = `manual`. Excluded from pace PRs.

12. Nutrition module

12.1 Data sources

Source	Use	Confidence label
USDA FoodData Central (Foundation + SR Legacy subset, ~1–2k common foods), imported into `foods` during seeding	Generic foods (chicken breast, rice, eggs, oats, skyr-type yoghurt…)	reference
User custom foods	Anything else	user
Open Food Facts (V1.1)	Packaged foods by barcode/search	crowdsourced
TheMealDB	Recipe inspiration (name, image, ingredients text, instructions)	none (nutrition unavailable)
User recipes	Recipes built from foods with grams	calculated

USDA FDC data is public domain (CC0); still show "Food data: USDA FoodData Central" in Data sources. Store nutrition per 100 g (or per 100 ml for liquids) plus named portions (`food_portions`: "1 large egg = 50 g").

12.2 Logging

- Search (catalogue + custom, recent and favourites first), select portion or enter grams/ml, choose meal type and date (default: now's local date and meal by time of day — before 10:30 breakfast, 10:30–15:00 lunch, 15:00–17:00 snack, 17:00–21:30 dinner, after that snack).
- Quick add: name + kcal (+ optional macros).
- Copy meal from another day, copy yesterday's meal, save meal as "saved meal", log saved meal.
- **Saved meals** ("my usual breakfast") can be built from scratch, edited, duplicated and removed. Items are foods (grams), the user's recipes (servings) or quick items (name + own numbers); each keeps a nutrition snapshot that scales exactly with its amount. "Save as a meal" opens the editor pre-filled with that meal's entries. "Log it" logs all items or only the ticked ones.
- Every item stores a snapshot: name, quantity, unit, grams, kcal, protein, carbs, fat, fibre, source, confidence.
- Items with unknown macros show a "Partial data" badge; day totals show "Some items lack full nutrition data".
- "Mark day complete" toggle (used for calibration and reports). Auto-suggested in the evening if ≥ 2 meals logged, never auto-set.
- Logging allowed for today and past dates; future dates only via the meal planner.

12.3 Recipes (TheMealDB)

- Browse: search by name, category list, filter by main ingredient, random. Results cached in `recipes` (unique `source + external_id`) via `recipes-proxy`.
- Normalisation (`recipe-normalizer.ts`): map `strIngredient1..20` + `strMeasure1..20` to ingredient lines, drop empty pairs, parse measures (12.4), split instructions into steps, tags, area (cuisine), category, source URL, YouTube link.
- TheMealDB has **no servings and no nutrition**. `servings_stated = null`, `nutritional_confidence = 'none'`. Recipe detail shows "Nutrition unavailable — log this meal with your own estimate or rebuild it as a PACE recipe".
- Scaling requires a base. If `servings_stated` is null, ask "How many servings does this recipe make?" (saved per user in `saved_recipes.base_servings`). Until answered, quantities are shown unscaled.
- Attribution: "Recipe from TheMealDB" + link to the original on every recipe screen.
- Dietary filtering is keyword-based on ingredient names (meat/fish list for vegetarian, animal products for vegan, allergy keywords) and shows "Always check ingredients yourself".

12.4 Measure parser and scaling (domain/nutrition)

- Parse: integers, decimals (comma or dot), fractions `1/2`, mixed `1 1/2`, unicode ½ ¼ ¾ ⅓ ⅔, ranges `2-3`, units with aliases (g, gram(s); kg; ml; l, litre/liter; tsp, teaspoon; tbsp, tbs, tablespoon; cup(s); oz; lb; pinch; clove(s); can/tin), number glued to unit (`175g`).
- Non-scalable: "to taste", "to serve", "garnish", "dash", "drizzle", "handful", "sprinkling", "as required", empty → `parse_status = to_taste/unparsed`, shown verbatim.
- Scale factor = target servings / base servings. Ranges scale both ends. Count units (eggs, cloves) rounded to nearest ¼ and marked "approx.". Metric weights rounded to 5 g above 50 g, 1 g below. Volumes are never converted to grams (density unknown).
- Per-person nutrition is shown only when the recipe has nutrition (user recipes): `per_serving = total / servings`.

12.5 User recipes

Created, edited and duplicated in the recipe editor (`recipe-edit`). A copy of a TheMealDB recipe ("Make it my own") keeps its ingredients as text, its method and its link, and has no nutrition until the user adds it.

- **Ingredients** are either foods (catalogue/custom, grams; a per-100 g snapshot is stored with the ingredient) or free text: a name plus an optional amount ("2 cups", "to taste") parsed with 12.4 so it scales and reaches the shopping list. Free-text ingredients never get nutrition.
- **Nutrition per serving** is either calculated (Σ food per-100 g × grams / 100 ÷ servings; confidence `calculated`, shown as "Calculated from N of M ingredients" when some are free text) or typed in by the user (confidence `user_estimate`, kcal required, macros optional, shown as "Your estimate"). With neither, confidence is `none` ("Nutrition unavailable").
- Name, description, servings 1–50, prep/cook minutes, up to 8 tags, up to 40 steps, an optional https source link. Ingredients and steps can be reordered with buttons.
- Logging: servings in ¼ steps, today or the two days before, any meal. Logged entries are snapshots; editing a recipe never changes history.
- Your own foods can be edited and removed (archived) the same way.

User recipes with nutrition are the recipes the meal planner uses for macro-aware suggestions.

12.6 Meal planner (V1 simple)

Week grid (days × meal types). Add a saved meal, user recipe, TheMealDB recipe or custom text. Shows planned kcal/macros per day where nutrition is known (otherwise "partial"). Suggestions: rank user recipes and saved meals by closeness to the slot's share of the daily target (target × slot share: breakfast 25%, lunch 35%, dinner 30%, snacks 10%, adjusted by meals per day), excluding allergens and disliked foods, respecting cooking time preference. "Mark eaten" creates a meal log with the planned portion (editable). Regenerate suggestions = next ranked options.

12.7 Shopping list

Generate from a date range of planned meals × chosen servings. A saved meal contributes its food items (grams) and quick items (as text); a recipe inside a saved meal contributes that recipe's ingredients scaled by its servings in the meal × planned servings. Merge lines with the same normalised ingredient name **and** same unit; different units stay separate lines; unparsed measures become text lines with recipe name. Check/uncheck works offline. Manual items can be added.

12.8 Water

Quick add buttons (250 ml, 500 ml, custom), undo for 5 s, daily bar vs target. Entries > 2000 ml at once or day total > 10 L require confirmation.

13. Weight and body progress

13.1 Weight logging

Fields: weight, date/time (default now), note, source (manual). Edit and delete with confirmation. Multiple entries per day allowed. Tip: "Weigh yourself in the morning, after the toilet, before eating, in similar clothing."

Validation: 30–350 kg; not in the future; not before date of birth. If the value differs from the latest 7-day average by > 3 kg (or > 3%), ask "Is this correct?" (catches typos such as 180 instead of 108).

13.2 Chart

- Daily values are joined into a thin line (with a dot per day while ≤ 45 days are in view), so the ups and downs are visible; a stretch of more than 7 days without data is joined with a **dashed** segment, never drawn as if it were measured. On top: the 7-day rolling average = mean of daily values within [d−6, d], shown only when ≥ 3 daily values in the window (thicker, accent).
- **Scale:** faint grid lines at round values (`niceDomain`: steps of 1/2/5/10, e.g. 85 · 90 · … · 110 for target 87 and start 108), **numbers on the left axis**. The goal's start and target (and 2nd target) are **pale full-width lines** (start solid, targets dashed) labelled at their right end ("Start 108.0", "Target 90.0"), labels nudged apart when lines are close. The y-range runs from the target (bottom) to about the start (top) and grows when a weight passes either (`weightYDomain`; a muscle-gain goal flips it).
- Ranges (calendar periods): **Week · Month · Year · All**, default Month. The axis runs from the first weigh-in inside the period (week start from the profile; at least one week shown) to today, so the data always fills the chart. The overview cards (Home, Progress) keep a rolling 30-day window.
- Time axis labels follow the span: weekdays ("Mon") within one week, dates ("4 Nov") up to 3 weeks, Mondays ("3 Nov") up to ~2 months, months ("Feb", "Jan 2027" at a new year) up to 2 years, then years; never more labels than fit.
- Tap or drag along the chart: a guide line and highlighted dot show the date, value and 7-day average. A horizontal drag owns the gesture once started (gesture handler), so it can't stop halfway; a vertical drag scrolls the screen. The chart fades in when the range changes.
- Gaps > 7 days: average line breaks (no interpolation).
- Projection band (Section 8.8) shown for future dates when available.

13.3 Statistics

Tiles. Progress tab, under the chart: Current weight (latest day), 7-day average, Start, Total change. Weight details adds Target, Last 7 days and Last 30 days. Change tiles show an arrow and a goal-aware tint (`changeTone`): toward the goal (down for lose weight, up for build muscle) = green (successSurface + accent); away from it = **amber** (warningSurface + warning, never red: 23.4); maintain, general fitness or |change| < 0.1 kg = neutral. The spoken label says "toward / away from your goal".

Change since start (from goal starting weight to current 7-day average), change last 7 days and last 30 days (difference of 7-day averages, needs data at both ends), % of goal completed = (start − current_avg) / (start − target), clamped to 0–100%, estimated target date range (Section 8.8; shown only with a downward trend and enough data, otherwise "Not enough data yet"). Display to 1 decimal place; percentages to whole numbers.

13.4 Goal reached

When the 7-day average reaches the target: celebration, then offer (a) set the secondary target, (b) switch to maintenance (proposes a new target = observed TDEE or TDEE), (c) set a new goal. Do not keep a deficit running silently.

13.5 Body measurements

Types: waist, hips, chest, arm (left/right), thigh (left/right), calf (left/right), neck. 10–250 cm. History list + line chart per type, change since first entry. Tip on measuring consistently (same time, relaxed, tape level).

13.6 Progress photos (V1.1)

Front/side/back, private bucket, signed URLs (60 min), never public, side-by-side compare, permanent delete (removes the storage object immediately). Max 5 MB, JPEG/PNG/HEIC converted to JPEG, EXIF location stripped before upload.

14. Milestones and personal records

14.1 Principles

Evaluated in `src/domain/milestones` after relevant writes (weight saved, session completed, activity saved, day complete). Unlocks are upserted with unique `(user_id, milestone_key)` so they are idempotent across devices and retries. Never revoked. Not re-awarded. When several unlock at once (e.g. importing history), show one summary ("You unlocked 4 milestones"). Weight thresholds use the **7-day average**, not a single weigh-in. No milestones for rate of loss, low intake or extreme volume.

14.2 Catalogue

See Section 25.5. Weight milestones are generated from the goal: first 1, 3, 5 kg lost, then every 5 kg lost; every round-number weight crossed that is a multiple of 5 below the start (e.g. 105, 100, 95, 90); target and secondary target. Changing the goal regenerates locked milestones; unlocked ones stay.

14.3 Personal records

Record	Rule
Longest walk / run / cycle	Max distance, GPS quality good/fair or manual with distance (manual labelled)
Fastest run pace	Best average pace for runs ≥ 1 km, GPS quality good/fair only
Most workout minutes in a week	Sum of session durations per local week
Push-up max reps	Max reps in a single set of any push-up variant (per variant)
Plank longest hold	Max duration of a plank set
Strength progress	Per exercise: best reps at a given weight; best total volume in a session

Records store the source session/activity; deleting the source recalculates the record.

15. Daily coach (rule-based recommendations)

15.1 Engine

`evaluate(context, now) → Recommendation[]`. Context: profile, goal, targets, today's and tomorrow's schedule, last 28 days of sessions/activities, last 21 days of weights, today's nutrition and water, latest check-in, active injuries, dismissed cards. Each recommendation: `id, rule_id, priority (0 = highest), category, title, body, reason ("Why am I seeing this?"), action (optional deep link), expires_at`. Dashboard shows the top 3. Safety rules suppress conflicting training suggestions. Dismissed cards hide until the next day. Rules are pure functions with unit tests.

15.2 Rule set V1

ID	Pri	Condition	Message (example)
S1	0	Active/recovering injury and today's plan contains restricted exercises	"Today's workout was adjusted for your ankle. Restricted exercises were swapped."
S2	0	Check-in pain ≥ 4/10	"You reported pain today. Rest that area and consider professional advice if it persists."
S3	0	A game or practice today/tomorrow of a sport the injury restricts (status active/recovering/cleared_training)	"Basketball game tomorrow, but your ankle is marked as recovering. Check with a professional before playing." (the sport's own words)
S4	1	Logged intake < floor on 3 of the last 4 complete days	"Your intake has been well below your target. Eating enough supports training and recovery."
R1	2	Match tomorrow	"Match tomorrow. Keep today light: mobility or an easy walk, hydrate, carbs with meals, good sleep."
R2	2	Match yesterday	"Recovery day after your match. An easy 20–30 min walk and mobility help."
R3	2	Check-in fatigue ≥ 4 or sleep < 6 h, and a hard session planned	"You're tired today. Swap to the 20-minute version or a recovery session."
R4	2	Two hard days in a row completed and a hard session planned today	"Two hard days in a row. Today is better as an easy day."
R5	3	Load spike (8.11)	"Your training load jumped this week. Consider an easier day."
T1	3	Planned workout not started by its time + 2 h	"Short on time? The 20-minute version covers the essentials."
T2	4	Missed workout yesterday	"Missed yesterday's session? Move it or skip — no problem." (action: Move)
N1	4	After 18:00, protein remaining ≥ 30% of target	"35 g protein to go. Skyr, eggs, chicken or tuna are easy options."
N2	5	After 15:00, water < 50% of target	"Halfway through the day — time for some water."
N3	5	After 14:00, no meals logged today	"Don't forget to log lunch."
W1	5	No weigh-in for 4 days (if weight goal active)	"Log a morning weigh-in to keep your trend accurate."
W2	3	Recalibration proposal pending	"Your 2-week review is ready."
C1	6	Completed ≥ planned sessions this week	"3 of 3 workouts done this week. Great consistency."
C2	6	New user (< 3 days data)	Onboarding tips (log weight, try a workout)

Never: diagnose, recommend training through pain, suggest cutting calories, override professional restrictions, or recommend a hard session on MD-1.

With nutrition switched off (6.5), S4, N1–N3 and W2 are never shown (`NUTRITION_RULES`).

16. Reports

Weekly report (local week, default Monday–Sunday), generated in the domain module and cached in `weekly_reports` as a JSON snapshot when the week ends (current week shown live and labelled "In progress").

Contents: weekly average weight (start of week vs end), trend, sessions completed vs planned (completion %), total training minutes, training load, activities count, walking/running/cycling distance, average kcal and protein **on complete days** with "based on N of 7 days", water average on logged days, milestones unlocked, PRs set. Every metric distinguishes "no data" from zero.

17. Notifications (local, V1)

Type	Default	Notes
Workout reminder	On, 60 min before scheduled time (or 09:00 if no time)	Not on rest days
Match reminder	On, evening before (19:00)	
Weigh-in reminder	Off	Morning at chosen time
Meal logging reminder	Off	Up to 3 per day at chosen times
Water reminder	Off	Max 3 per day
Weekly report	On, Monday 09:00	
Rest timer	On during workouts	

Meal and water reminders are never scheduled while nutrition is switched off (6.5).

Rules: quiet hours 22:00–07:00; global cap 4 per day (rest timer excluded); notification permission asked only when the user turns on the first reminder. Reconcile on app start and after any schedule/preference/timezone change: cancel all PACE-scheduled notifications and schedule the next 14 days, staying under iOS's 64 pending notification limit. Notification text never includes weight or health details (lock screen privacy).

18. Offline and synchronisation

18.1 Scope

Offline read: cached plan, scheduled workouts, templates, exercise library, recent foods, saved meals, saved recipes, last 90 days of weights, targets. Offline write: weight, measurements, water, meal logs and items, workout sessions and sets, activities and routes, check-ins, shopping list checks, schedule moves.

18.2 Mechanism

- TanStack Query cache persisted to storage for reads.
- Writes go to the local SQLite **outbox**: `(id, user_id, entity, record_id, op: upsert|delete, payload, created_at, attempts, last_error, status)`. The UI updates optimistically.
- Sync engine runs on app foreground, network regain, after each write (debounced 2 s) and every 5 min while foregrounded. FIFO per entity, parents before children (meal_log before items, session before sets, activity row before route upload).
- Server writes are `upsert` on the client UUID (idempotent). Conflict resolution: last-write-wins by `client_updated_at`, tie-break by device id; append-only logs (weights, water, sets) never conflict.
- Ops are collapsed: create + delete before sync → nothing sent; multiple updates → last one.
- After an op reaches the server it stays applied to local reads for 15 s (or until the refetch lands), so a just-saved change never flashes the old values. It never counts as pending in the sync status.
- Errors: network/5xx → exponential backoff (2 s → 5 min cap); 401 → refresh session and retry; validation/RLS (4xx) → status `failed`, shown in Sync status with Retry, Edit or Delete. Never dropped automatically.
- Outbox and local caches are scoped to `user_id`. Sign-out with pending items shows a warning; the data of user A is never synced under user B. Sign-out clears local caches after confirmation.
- Sync status indicator: synced / syncing / N pending / N failed.

19. Security and privacy

- RLS enabled on every table in `public` (Section 21). Default deny.
- Mobile client uses only the Supabase URL and publishable (anon) key. Service role key exists only in Edge Function secrets.
- Create the Supabase project in an **EU region** (GDPR, users in Europe).
- Edge Functions validate input with zod, authenticate the caller from the JWT, and rate-limit (`recipes-proxy`: 60 requests/min per user, cached responses 24 h).
- Storage buckets private; object path starts with `{user_id}/`; size and MIME limits enforced by bucket config.
- Health data (injuries, conditions, check-in pain) is special-category data under GDPR Art. 9: collected only after explicit consent, optional, deletable independently ("Delete all health information").
- Location: consent at first use, routes deletable, never in analytics/logs.
- Account deletion in-app (required by App Store): re-authenticate (password), confirm by typing DELETE, call `delete-account` → deletes Storage objects under the user prefix, then `auth.admin.deleteUser` (all tables cascade), then clears local data. Idempotent and retryable. Backups expire according to the provider's retention, stated in the privacy policy.
- Data export: client-side JSON export of all user tables (+ routes as JSON, GPX in V1.1), shared via the OS share sheet.
- No personal data in logs, crash reports or analytics. No third-party ad or tracking SDKs.
- Privacy policy, terms, disclaimer and data-source licences reachable from onboarding and Profile.

20. Database schema (V1)

The DDL below is condensed pseudo-SQL: `...` stands for the standard columns (`created_at`, `updated_at`, `client_updated_at`), and expression-based uniqueness (e.g. `coalesce(...)`) must be implemented as unique indexes. Conventions: `id uuid primary key` (client-generated for user data; `default gen_random_uuid()` as fallback); `user_id uuid not null references auth.users(id) on delete cascade` on **every** user-owned table, including child tables (simpler, faster RLS); `created_at timestamptz not null default now()`; `updated_at timestamptz not null default now()` maintained by a trigger; `client_updated_at timestamptz` on syncable tables; enums as `text` + `check` constraints (easier migrations). Index `(user_id, <date column>)` on all time-series tables.

20.1 Identity and preferences

```
profiles (
  id uuid pk references auth.users on delete cascade,
  display_name text check (char_length(display_name) between 1 and 50),
  date_of_birth date,
  calc_sex text check (calc_sex in ('male','female')),
  height_cm numeric(5,1) check (height_cm between 120 and 230),
  unit_system text not null default 'metric' check (unit_system in ('metric','imperial')),
  timezone text not null default 'UTC',
  activity_level text check (activity_level in ('sedentary','light','moderate','very','extra')),
  average_daily_steps int check (average_daily_steps between 0 and 60000),
  week_start smallint not null default 1 check (week_start in (1,7)),   -- ISO weekday
  onboarding_step text,
  onboarding_completed_at timestamptz,
  created_at, updated_at,
  check (onboarding_completed_at is null or
         (display_name is not null and date_of_birth is not null and calc_sex is not null
          and height_cm is not null and activity_level is not null))
)
-- trigger on auth.users insert creates an empty profiles row

user_goals (
  id, user_id,
  goal_type text check (goal_type in ('lose_weight','maintain','build_muscle','general_fitness')),
  focus_tags text[] not null default '{}',     -- strength, sport, running, cardio, mobility ('futsal' from old apps is stor
  starting_weight_kg numeric(5,2), target_weight_kg numeric(5,2), secondary_target_weight_kg numeric(5,2),
  weekly_rate_pct numeric(4,2) check (weekly_rate_pct between 0 and 1.0),
  target_date date,
  status text check (status in ('active','achieved','paused','abandoned')),
  started_at date not null, ended_at date, created_at, updated_at
)
unique index one_active_goal on user_goals(user_id) where status = 'active'

user_preferences (
  user_id uuid pk references auth.users on delete cascade,
  diet_type text check (diet_type in ('omnivore','vegetarian','vegan','pescatarian','other')),
  allergies text[] default '{}', intolerances text[] default '{}', disliked_foods text[] default '{}',
  preferred_cuisines text[] default '{}',
  meals_per_day smallint check (meals_per_day between 2 and 6) default 3,
  cooking_time_max_minutes smallint,          -- null = any
  household_servings smallint check (household_servings between 1 and 8) default 1,
  training_days_per_week smallint check (training_days_per_week between 0 and 7),
  preferred_training_days smallint[] default '{}',   -- ISO weekdays 1..7
  max_session_minutes smallint check (max_session_minutes between 15 and 120) default 45,
  preferred_training_time time,
  training_environment text check (training_environment in ('home','gym','outdoors','mixed')),
  notification_preferences jsonb not null default '{}',
  sports text[] not null default '{}',  -- sports catalogue keys, max 8; empty = no sport
  nutrition_enabled boolean not null default true,  -- 6.5; off hides food, calories and water
  locale text default 'en', updated_at
)

user_equipment (id, user_id, equipment_type text, custom_name text, max_weight_kg numeric(5,1),
  unique (user_id, equipment_type, coalesce(custom_name,'')))

consents (id, user_id, consent_type text check (consent_type in
  ('terms','privacy','health_data','location','background_location','notifications')),
  document_version text, granted boolean, granted_at timestamptz, revoked_at timestamptz)
```

20.2 Health and check-ins

```
health_conditions (
  id, user_id,
  kind text check (kind in ('injury','condition')),
  body_area text, side text check (side in ('left','right','both','na')),
  description text,
  status text check (status in ('active','recovering','cleared_training','cleared_sport','resolved')),
  pain_level smallint check (pain_level between 0 and 10),
  restriction_tags text[] not null default '{}',     -- derived from area + manual additions
  professional_restrictions text,
  professional_clearance boolean default false,
  onset_date date, resolved_at date, created_at, updated_at
)

daily_checkins (id, user_id, local_date date, sleep_hours numeric(3,1) check (sleep_hours between 0 and 24),
  fatigue smallint check (fatigue between 1 and 5), soreness smallint check (soreness between 1 and 5),
  pain smallint check (pain between 0 and 10), pain_area text, note text,
  unique (user_id, local_date))
```

20.3 Body tracking

```
weight_entries (id, user_id, weight_kg numeric(5,2) check (weight_kg between 30 and 350),
  measured_at timestamptz not null, local_date date not null, note text check (char_length(note) <= 500),
  source text default 'manual', client_updated_at, created_at, updated_at)
index (user_id, measured_at desc)

body_measurements (id, user_id, measurement_type text check (measurement_type in
  ('waist','hips','chest','arm_left','arm_right','thigh_left','thigh_right','calf_left','calf_right','neck')),
  value_cm numeric(5,1) check (value_cm between 10 and 250), measured_at timestamptz, local_date date, ...)
index (user_id, measurement_type, measured_at desc)

progress_photos (V1.1) (id, user_id, pose text check (pose in ('front','side','back')),
  storage_path text, taken_at timestamptz, local_date date, ...)
```

20.4 Training

```
exercises (id, owner_id uuid null references auth.users on delete cascade,   -- null = system
  slug text unique, name text not null, description text,
  category text check (category in ('strength','mobility','core','conditioning','football','sport_skills','recovery','cardio
  sports text[] not null default '{}',  -- sports a drill belongs to (catalogue keys); empty = everyone
  muscle_groups text[], equipment text[], difficulty text check (difficulty in ('beginner','intermediate','advanced')),
  tracking_type text check (tracking_type in ('reps','reps_weight','time','distance')),
  is_unilateral boolean default false, instructions text[], caution_notes text,
  contraindication_tags text[] not null default '{}', media_url text,
  default_sets smallint, default_reps_min smallint, default_reps_max smallint, default_duration_s smallint,
  met numeric(4,1), archived_at timestamptz, created_at, updated_at)

workout_templates (id, owner_id null, slug text unique, name, category, intensity text check (intensity in ('easy','moderate
  estimated_minutes smallint, short_version_of uuid null references workout_templates, description, ...)

workout_template_exercises (id, template_id, exercise_id, order_index, sets, reps_min, reps_max,
  duration_s, rest_s, notes, alternative_exercise_id uuid null references exercises)

training_plans (id, user_id, name, goal_type, start_date, end_date null, status text check (status in ('active','archived'))
  generator_version text, generated_from jsonb, created_at, updated_at)
unique index one_active_plan on training_plans(user_id) where status = 'active'

recurring_events (id, user_id, training_plan_id null, title, category text check (category in
  ('workout','run','walk','cycle','other') or category ~ '^[a-z_]{2,30}_(match|practice)$'),
  template_id null, weekday smallint check (weekday between 1 and 7), local_time time, timezone text,
  duration_minutes smallint, intensity text, starts_on date, ends_on date null, active boolean default true, ...)

scheduled_workouts (id, user_id, training_plan_id null, recurring_event_id null, template_id null,
  title, category, intensity, scheduled_date date not null, scheduled_time time null, timezone text,
  estimated_duration_minutes smallint, status text check (status in ('planned','completed','skipped','cancelled')),
  is_user_modified boolean default false, slot smallint default 0, notes, client_updated_at, ...)
unique (recurring_event_id, scheduled_date) where recurring_event_id is not null
unique (training_plan_id, scheduled_date, slot) where training_plan_id is not null and recurring_event_id is null
index (user_id, scheduled_date)

scheduled_workout_exercises (id, user_id, scheduled_workout_id on delete cascade, exercise_id,
  order_index, sets, reps_min, reps_max, duration_s, rest_s, suggested_weight_kg, alternative_exercise_id, notes)

workout_sessions (id, user_id, scheduled_workout_id null on delete set null, template_id null, title, category,
  started_at timestamptz, ended_at timestamptz, local_date date, duration_seconds int, paused_seconds int default 0,
  perceived_exertion numeric(3,1) check (perceived_exertion between 0 and 10),
  feeling smallint check (feeling between 1 and 5), total_volume_kg numeric(8,1),
  calories_estimated int, completion_status text check (completion_status in ('completed','partial','abandoned')),
  source text default 'app', notes, client_updated_at, ...)
index (user_id, started_at desc)

exercise_sets (id, user_id, workout_session_id on delete cascade, exercise_id, set_number smallint,
  repetitions smallint check (repetitions between 0 and 500), weight_kg numeric(6,2) check (weight_kg between 0 and 500),
  duration_seconds int, completed boolean, skipped boolean default false, pain_flag boolean default false)
```

20.5 Activities

```
activities (id, user_id, activity_type text check (activity_type in ('walk','run','cycle','other')),
  started_at timestamptz, ended_at timestamptz, local_date date,
  elapsed_seconds int, moving_seconds int, distance_meters int check (distance_meters >= 0),
  average_speed_mps numeric(6,3), max_speed_mps numeric(6,3), average_pace_seconds_per_km int,
  calories_estimated int, gps_quality text check (gps_quality in ('good','fair','poor','none')),
  points_total int, points_accepted int, route_storage_path text, route_deleted_at timestamptz,
  source text check (source in ('gps','manual')), notes, client_updated_at, ...)
index (user_id, started_at desc)
```

20.6 Nutrition

```
foods (id, owner_id null, name text not null, brand text, barcode text,
  basis text check (basis in ('per_100g','per_100ml')),
  kcal numeric(6,1), protein_g numeric(6,2), carbs_g numeric(6,2), fat_g numeric(6,2), fibre_g numeric(6,2),
  source text check (source in ('usda','off','user')), external_id text,
  nutritional_confidence text check (nutritional_confidence in ('reference','crowdsourced','user')),
  archived_at, created_at, updated_at,
  unique (source, external_id))
index foods using gin (to_tsvector('simple', name)); index (barcode)

food_portions (id, food_id on delete cascade, label text, grams numeric(7,2) check (grams > 0))
favorite_foods (user_id, food_id, created_at, primary key (user_id, food_id))

recipes (id, owner_id null, source text check (source in ('themealdb','user')), external_id text,
  name, description, image_url, category, cuisine, tags text[], instructions text[],
  preparation_minutes, cooking_minutes, servings_stated smallint null, source_url, video_url,
  nutritional_confidence text check (nutritional_confidence in ('none','calculated','user_estimate')),  -- user_estimate: ow
  kcal_per_serving, protein_per_serving, carbs_per_serving, fat_per_serving,   -- null unless calculated
  fetched_at timestamptz, created_at, updated_at,
  unique (source, external_id))

recipe_ingredients (id, recipe_id on delete cascade, order_index, ingredient_name, raw_measure text,
  quantity_min numeric, quantity_max numeric, unit text,
  parse_status text check (parse_status in ('parsed','unparsed','to_taste')),
  food_id null, grams numeric(7,2) null, preparation_note)

saved_recipes (user_id, recipe_id, base_servings smallint null, default_servings smallint, note, created_at,
  primary key (user_id, recipe_id))

meal_logs (id, user_id, local_date date, meal_type text check (meal_type in ('breakfast','lunch','dinner','snack')),
  title, notes, client_updated_at, ...)
index (user_id, local_date)

meal_log_items (id, user_id, meal_log_id on delete cascade, food_id null, recipe_id null,
  name_snapshot text not null, quantity numeric, unit text, grams numeric(7,2) null,
  kcal numeric(7,1) null, protein_g numeric(6,2) null, carbs_g numeric(6,2) null, fat_g numeric(6,2) null,
  fibre_g numeric(6,2) null, source_snapshot text, confidence_snapshot text, client_updated_at, ...)
view meal_log_totals / daily_nutrition_totals (sum, plus count of items with null macros)

saved_meals (id, user_id, name, meal_type, ...)  saved_meal_items (id, user_id, saved_meal_id, ...same snapshot fields)

day_log_status (user_id, local_date, nutrition_complete boolean default false, updated_at,
  primary key (user_id, local_date))

water_logs (id, user_id, amount_ml int check (amount_ml between 1 and 5000), recorded_at timestamptz, local_date date, ...)

nutrition_targets (id, user_id, effective_from date not null, kcal int, protein_g int, carbs_g int, fat_g int,
  fibre_g int, water_ml int, bmr numeric, tdee numeric, floor_kcal int, limited_by_floor boolean,
  is_user_edited boolean, calculation_version text, calculation_inputs jsonb, created_at,
  unique (user_id, effective_from))
-- target for date D = row with max(effective_from) <= D

target_adjustments (id, user_id, proposed_at timestamptz, window_start date, window_end date,
  current_kcal int, proposed_kcal int, observed_tdee numeric, actual_rate_kg_week numeric, adherence numeric,
  reason_code text, status text check (status in ('proposed','accepted','rejected','superseded')),
  decided_at timestamptz, resulting_target_id uuid null, inputs jsonb)

meal_plan_entries (id, user_id, local_date, meal_type, recipe_id null, saved_meal_id null, custom_title text null,
  servings numeric(4,1), status text check (status in ('planned','eaten','skipped')), meal_log_id null, ...)

shopping_lists (id, user_id, name, range_start date, range_end date, status text check (status in ('active','archived')), ..
shopping_list_items (id, user_id, shopping_list_id on delete cascade, ingredient_name, quantity_min, quantity_max,
  unit, raw_text, source_recipe_ids uuid[], is_checked boolean default false, is_manual boolean default false, ...)
```

20.7 Progress and reports

```
user_milestones (id, user_id, milestone_key text, title, description, threshold numeric null,
  unlocked_at timestamptz, value numeric null, metadata jsonb, unique (user_id, milestone_key))

personal_records (id, user_id, record_type text, exercise_id null, value numeric, unit text,
  achieved_at timestamptz, source_type text check (source_type in ('session','activity','week')), source_id uuid,
  is_current boolean default true, created_at)
unique index on personal_records(user_id, record_type, coalesce(exercise_id, '00000000-0000-0000-0000-000000000000'::uuid))

weekly_reports (id, user_id, week_start date, payload jsonb, generated_at timestamptz, unique (user_id, week_start))
```

20.8 Migrations

One concern per migration file (`supabase/migrations/YYYYMMDDHHMMSS_description.sql`), never edit an applied migration, regenerate `database.types.ts` after each schema change (`supabase gen types typescript`). System seed data (exercises, templates, foods) in idempotent seed migrations using `on conflict (slug|source,external_id) do update`. Development-only seed (example user) lives in `supabase/seed/dev.sql` and is never applied to production.

21. Row Level Security and Storage policies

21.1 Standard user-owned table

```
alter table public.weight_entries enable row level security;

create policy "select own" on public.weight_entries for select to authenticated
  using ((select auth.uid()) = user_id);
create policy "insert own" on public.weight_entries for insert to authenticated
  with check ((select auth.uid()) = user_id);
create policy "update own" on public.weight_entries for update to authenticated
  using ((select auth.uid()) = user_id) with check ((select auth.uid()) = user_id);
create policy "delete own" on public.weight_entries for delete to authenticated
  using ((select auth.uid()) = user_id);
```

Apply to every table with `user_id`. `profiles` uses `id` instead of `user_id`. Child tables also verify the parent belongs to the user in `with check` (e.g. `exists (select 1 from meal_logs m where m.id = meal_log_id and m.user_id = (select auth.uid()))`).

21.2 Shared catalogue tables (exercises, workout_templates, foods, recipes)

```
-- read: system rows or own rows
using (owner_id is null or owner_id = (select auth.uid()))
-- insert/update/delete: own rows only
with check (owner_id = (select auth.uid()))
```

System rows (`owner_id is null`) are written only by migrations or the `recipes-proxy` function (service role). Child tables (`food_portions`, `recipe_ingredients`, `workout_template_exercises`) inherit visibility from the parent via `exists`.

21.3 Storage

Buckets: `routes` (private, 2 MB, application/gzip, application/json), `avatars` (private, 2 MB, image/jpeg, image/png), `progress-photos` (V1.1, private, 5 MB, image/jpeg).

```
create policy "own folder" on storage.objects for all to authenticated
  using (bucket_id in ('routes','avatars','progress-photos')
         and (storage.foldername(name))[1] = (select auth.uid())::text)
  with check (bucket_id in ('routes','avatars','progress-photos')
         and (storage.foldername(name))[1] = (select auth.uid())::text);
```

21.4 RLS tests (pgTAP, required)

For every user-owned table: user A can CRUD own rows; user B cannot select, update or delete A's rows (0 rows affected / empty result); B cannot insert a row with `user_id = A`; anonymous role sees nothing. For catalogue tables: both users see system rows; neither sees the other's custom rows; neither can modify system rows. For storage: B cannot read or write under A's folder.

22. Integrations

22.1 Provider interfaces

```
interface RecipeProvider {
  id: 'themealdb';
  search(query: string): Promise<RecipeSummary[]>;
  getById(externalId: string): Promise<NormalizedRecipe | null>;
  random(): Promise<NormalizedRecipe>;
  categories(): Promise<string[]>;
  byCategory(category: string): Promise<RecipeSummary[]>;
  byIngredient(ingredient: string): Promise<RecipeSummary[]>;
}
interface FoodProvider {            // usda-catalogue (local DB) in V1, openfoodfacts in V1.1
  search(query: string, opts: { limit: number }): Promise<NormalizedFood[]>;
  getByBarcode?(barcode: string): Promise<NormalizedFood | null>;
}
```

All responses validated with zod before normalisation; invalid items are dropped and counted, never crash the UI.

22.2 TheMealDB

Endpoints (base `https://www.themealdb.com/api/json/v1/{key}/`): `search.php?s=`, `lookup.php?i=`, `random.php`, `categories.php`, `filter.php?c=`, `filter.php?i=`. The public test key `1` is for development/education only; production use needs a supporter key, kept as an Edge Function secret. Therefore the app calls `recipes-proxy` (Edge Function) which adds the key, caches responses 24 h, upserts normalised recipes into `recipes`, and rate-limits per user. In local development the proxy may use key `1`. Filter endpoints return only id/name/thumbnail; details are fetched on open. `meals: null` means no results (not an error).

22.3 Open Food Facts (V1.1)

Product endpoint `https://world.openfoodfacts.org/api/v2/product/{barcode}.json`. Must send a custom User-Agent `PACE/<version> (<contact email from config>)`. Respect rate limits (product reads ~100/min, search ~10/min — verify current limits). Data licence ODbL: show attribution. Map `nutriments.energy-kcal_100g`, `proteins_100g`, `carbohydrates_100g`, `fat_100g`, `fiber_100g`; if kcal missing but kJ present, kcal = kJ / 4.184. Missing fields stay null. Label "Crowdsourced — check the package". Cache scanned products into `foods` (source `off`).

22.4 Failure policy

External API failure shows cached data or an error with retry. Manual logging always works.

23. Design system

23.1 Tokens

Token	Dark (default)	Light
background	#121417	#F7F8FA
surface	#1B1E23	#FFFFFF
surfaceRaised	#23272E	#F0F2F5
border	#2E333B	#E1E4E8
textPrimary	#F2F4F7	#101318
textSecondary	#A3AAB5	#5A6270
accent (progress/positive)	#34D399	#047857
onAccent	#0B1F17	#FFFFFF
successSurface (done / unlocked cards, change toward the goal)	#1C3430	#E8F4F0
dangerSurface (field with a validation error)	#352628	#FCEFEE
warningSurface (change away from the goal)	#33291C	#FBF1E6
warning	#F5A524	#A3480A
danger	#F97066	#C4281C
info	#60A5FA	#2563EB
chart series	weight points #A3AAB5, average #34D399, target #60A5FA (dashed), projection band accent at 15% opacity	same hues, light variants

Spacing scale 4, 8, 12, 16, 20, 24, 32, 40. Radius 8 (inputs), 12 (buttons), 16 (cards), 24 (sheets). Typography: system font; sizes 12/14/16/18/22/28/34; tabular numerals for metrics. Elevation: avoid shadows in dark mode; use surfaceRaised. Motion: see 23.5.

23.2 Components (src/components/ui)

Button (primary, secondary, ghost, destructive; loading, disabled), IconButton, TextField, NumberField (unit-aware, locale decimal comma support), Select, SegmentedControl, Chip/ChipGroup, Toggle, Slider (RPE), DatePicker, TimePicker, Card, StatTile, ProgressRing, ProgressBar, ListItem, SectionHeader, BottomSheet, ConfirmDialog, Toast (with Undo), EmptyState, ErrorState, Skeleton, Badge (confidence, sync), Banner (warning/info), Tabs, ChartContainer (range selector + accessible summary).

Every button has a visible border (primary: accent, secondary and ghost: border, destructive: danger), so it always reads as tappable; ghost is an outline button.

Dates are picked from a calendar. `DateField` looks like the other inputs ("Tue, 10 Nov 2026" plus a calendar icon) and opens `CalendarSheet`, a bottom sheet:

- ‹ month › arrows or a sideways swipe change the month, and the grid slides.
- Tapping the month title jumps to a year list, then a month list (date of birth opens on the years).
- The week starts per profile. Today is outlined and the selected day filled.
- Days outside the allowed range are dimmed and can't be tapped. One tap picks a day and closes; there is also a "Today" shortcut.
- Allowed ranges: logs up to today; workout times up to 60 days back; date of birth for ages 18–100; planning or moving a workout from today.

Validation errors appear on the field. A field that is wrong eases (200 ms) into a red-tinted background (`dangerSurface`) with a red border, and the message drops in right under it (`FieldError`; no animation with reduce motion) (e.g. Log weight → Save without a weight: "Enter your weight" under Weight); the message disappears as soon as that field is edited (`useFieldErrors`). Banners are only for problems that aren't about one field (save failed, "Add at least one exercise"), and they slide in rather than pop.

Tappable cards say so. A card with `onPress` shows a small accent "Details ›" footer, or a specific label ("Weight details", "Start check-in", "Open training", "Open nutrition", "Change goal", "View"); list-like cards (scheduled workouts) show a

trailing chevron instead, like list rows. Recipe rows have a chevron too.

Card tones. `success` = done / unlocked (completed workouts, unlocked milestones, goal reached, fully done calendar days): successSurface fill, accent border and a 5 px accent bar on the left, so it is recognisable mid-scroll and not only by its icon. `muted` = locked / not yet available: background fill and a dashed border.

Text fields and the keyboard. The focused field is always scrolled above the keyboard and a screen's footer (Save/Continue) rides on top of it (`react-native-keyboard-controller`; Android edge-to-edge windows no longer resize for the keyboard). Return moves to the next field and keeps the keyboard open; a form's last field shows "done" and submits (never on destructive forms: there it only closes the keyboard). iOS shows a prev/next/done bar above the keyboard because number pads have no return key. Password and email fields are never auto-capitalised or auto-corrected, and password fields have a show/hide button.

Labels (buttons, segments, chips) never break inside a word: a one-word label stays on one line and shrinks to at most 85% on native if space is tight; longer labels wrap between words, at most two lines. Segments share their row by label length ("Breakfast" gets more room than "Any"). Checked at 360–384 dp widths.

23.3 Accessibility

All interactive elements have `accessibilityLabel` and role; touch targets ≥ 44 × 44 pt; contrast ≥ 4.5:1 for text (tokens above meet this on their backgrounds); support Dynamic Type / font scaling up to 200% without clipping key numbers; charts have a text summary ("Weight down 2.4 kg in 30 days"); never rely on colour alone (icons + labels for status).

23.4 Copy tone

Encouraging, neutral, never shaming. "No data yet" instead of "You failed to log". Weight gain is reported neutrally. Use the user's units everywhere.

23.5 Touch feedback, transitions and haptics

All values live in `motion` (`src/theme/tokens.ts`); components never hard-code timings.

What	Behaviour	Reduce motion on
Press (buttons, chips, segments, toggles)	`PressableScale`: spring to 0.97 scale and 0.9 opacity while the finger is down, back on release; no overshoot; native driver	opacity only
Press (cards, list rows)	same, softer scale 0.985	opacity only
Disabled	50% opacity, no press, no haptic	same
Screen push / pop	native stack `ios_from_right` on both platforms, 260 ms	140 ms fade
Modal screens (check-in)	slide up from the bottom	140 ms fade
Onboarding steps	200 ms cross-fade	140 ms fade
Tabs	subtle `shift`	none
List rows in editors, saved meals, weekly events	fade in 200 ms when added, slide out to the left with a fade (200 ms) when removed, slide 200 ms when others move (`AnimatedRow`, keyed by name so rows keep their animation when reordered)	no animation
Coach cards	swipe sideways to dismiss (or tap Dismiss): slides out 200 ms, then its height collapses 200 ms so the content below glides up; a short swipe springs back (`SwipeToDismiss`). The "Coach" header collapses with the last card	removed at once
Sheets and dialogs (calendar, confirmations)	`ModalShell`: no native Modal animation; a full-screen window (under status and navigation bars) whose content only appears once the window has its final size, then the backdrop fades in and the sheet slides up (dialogs zoom in), 260 ms; closing slides/fades out before unmounting	appear / disappear at once
Segmented controls	the selected highlight slides to the new option (200 ms) instead of jumping	jumps
Weight chart range change	chart fades in (200 ms)	none
Content getting shorter while scrolled down (e.g. a section hidden by a setting)	scroll glides back to the new end, so no blank area is left	same, instant
Toasts	fade/slide up in 200 ms, fade down out 140 ms; swipe sideways to clear	no animation

Haptics (`src/theme/haptics.ts`, expo-haptics): a selection tick for chips, new segments and toggles; a light tap for primary and destructive buttons; a success pulse after a save completes (`useSubmit`). Off on web and when reduce motion is on; failures are ignored.

24. Environment and configuration

24.1 Variables

Variable	Where	Purpose
EXPO_PUBLIC_SUPABASE_URL	app	Supabase project URL
EXPO_PUBLIC_SUPABASE_ANON_KEY	app	Publishable (anon) key — safe for clients; RLS protects data
EXPO_PUBLIC_APP_ENV	app	development / preview / production
EXPO_PUBLIC_OFF_CONTACT	app (V1.1)	Contact for Open Food Facts User-Agent
GOOGLE_MAPS_ANDROID_API_KEY	EAS secret → app config	Android map rendering (restrict key to the app's package + SHA-1)
SENTRY_DSN (optional)	app	Crash reporting
THEMEALDB_API_KEY	Edge Function secret	Recipe API key (`1` in dev)
SUPABASE_SERVICE_ROLE_KEY	Edge Function (provided automatically)	Never in the app or repo

`.env.example` lists all app variables with empty values. `.env*` files are git-ignored. `app.config.ts` reads variables and fails fast with a clear error if a required one is missing.

24.2 Environments

Separate Supabase projects for development and production (staging optional). Local development with Supabase CLI (`supabase start`) and Docker. Auth redirect URLs: `pace://auth/callback` plus the Expo dev URL scheme. Email confirmation on in production. EAS profiles: `development` (dev client, internal), `preview` (internal distribution), `production` (store).

25. Seed and reference data

25.1 Example user (development seed only)

Field	Value
Name	Dev User
Age / DOB	23 (DOB set so age = 23 on seed date)
Sex (calc)	male
Height	187 cm
Weight	108 kg
Goal	lose weight → 90 kg, secondary 87 kg, 0.5%/week
Focus	futsal, strength
Activity level	moderate (1.55)
Equipment	dumbbells (max 5 kg), kettlebell (5 kg), exercise mat
Max session	60 min (45 default)
Commitments	Futsal league match Saturday 18:00, 60 min, hard; futsal practice match Tuesday 20:00, 60 min, hard
Nutrition	omnivore, no restrictions, 3 meals + snacks, household 1 (2 available)
Injury	right ankle sprain, status recovering, pain 2/10

Expected calculations (must match tests):

Quantity	Value
BMR	10×108 + 6.25×187 − 5×23 + 5 = **2138.75**
TDEE	2138.75 × 1.55 = **3315.06**
Desired deficit	0.005 × 108 × 7700 / 7 = **594** kcal
Deficit applied	min(594, 750, 828.8) = 594
Target	round_to_10(2721.06) = **2720 kcal** (floor = max(1500, 2140) = 2140, not limiting)
Projected rate	(3315.06 − 2720) × 7 / 7700 = **0.54 kg/week**
Protein	1.8 × 90 = **162 g** (648 kcal)
Fat	0.25 × 2720 / 9 = 75.6 → **76 g** (684 kcal)
Carbs	floor((2720 − 648 − 684)/4) = floor(347) = **347 g** (1388 kcal)
Macro check	648 + 684 + 1388 = 2720 ✓
Water	35 × 108 = 3780 → **3800 ml** (round to 50)
Projection	18 kg / 0.54 ≈ 33 weeks (range 27–44 weeks) → "around 6–10 months"

Example generated week (ankle status = recovering):

Day	Session	Intensity	Notes
Mon	Strength A — Upper & core (45 min)	moderate	
Tue	Futsal practice match	hard	Caution banner while ankle is recovering
Wed	Recovery — easy walk 30 min + mobility 10 min	easy	MD+1 of Tue practice
Thu	Strength B — Lower & core, ankle-safe (45 min)	moderate	MD-2; jumping/lateral/single-leg balance excluded
Fri	Full rest (optional 15 min mobility)	easy	MD-1
Sat	Futsal league match	hard	Caution banner while ankle is recovering
Sun	Recovery walk 30–40 min	easy	MD+1

25.2 Exercise library seed (minimum)

Tags: I = impact, J = jumping, L = lateral, S = sprint, AH = ankle_load_high, SLB = single_leg_balance, DK = deep_knee_flexion, DH = deep_hip_flexion, HL = hinge_loaded, SP = spinal_loaded, O = overhead, PL = push_loaded, W = wrist_load.

Slug	Name	Category	Equipment	Tracking	Default	Contraindication tags
bodyweight-squat	Bodyweight squat	strength	none	reps	3×12–20	DK
goblet-squat	Goblet squat	strength	dumbbell/kettlebell	reps_weight	3×10–15	DK
box-squat	Squat to chair/box	strength	none	reps	3×10–15	—
reverse-lunge	Reverse lunge	strength	none/dumbbell	reps_weight	3×8–12/side	SLB, DK
split-squat	Split squat	strength	none/dumbbell	reps_weight	3×8–12/side	SLB, DK
step-up	Step-up	strength	bench/step	reps_weight	3×8–12/side	SLB, AH
db-rdl	Dumbbell Romanian deadlift	strength	dumbbell	reps_weight	3×10–15	HL
kb-deadlift	Kettlebell deadlift	strength	kettlebell	reps_weight	3×10–15	HL
glute-bridge	Glute bridge	strength	mat	reps	3×12–20	—
single-leg-glute-bridge	Single-leg glute bridge	strength	mat	reps	3×8–12/side	—
hip-thrust	Hip thrust	strength	bench, dumbbell	reps_weight	3×10–15	—
wall-sit	Wall sit	strength	none	time	3×30–45 s	DK
standing-calf-raise	Standing calf raise	strength	none/dumbbell	reps	3×12–20	AH
seated-calf-raise	Seated calf raise	strength	dumbbell	reps_weight	3×15–20	—
kb-swing	Kettlebell swing	conditioning	kettlebell	reps_weight	3×15–20	HL
push-up	Push-up	strength	none	reps	3×AMRAP	W, PL
incline-push-up	Incline push-up	strength	bench/table	reps	3×8–15	W
db-floor-press	Dumbbell floor press	strength	dumbbell, mat	reps_weight	3×12–15	PL
one-arm-db-row	One-arm dumbbell row	strength	dumbbell	reps_weight	3×10–15/side	—
bent-over-db-row	Bent-over dumbbell row	strength	dumbbell	reps_weight	3×12–15	HL
band-row	Band row	strength	resistance band	reps	3×12–15	—
band-pull-apart	Band pull-apart	strength	resistance band	reps	3×15–20	—
pull-up	Pull-up / assisted pull-up	strength	pull-up bar	reps	3×AMRAP	—
db-shoulder-press	Dumbbell shoulder press	strength	dumbbell	reps_weight	3×10–15	O
lateral-raise	Lateral raise	strength	dumbbell	reps_weight	3×12–20	—
db-curl	Dumbbell curl	strength	dumbbell	reps_weight	3×12–15	—
overhead-triceps	Overhead triceps extension	strength	dumbbell	reps_weight	3×12–15	O
plank	Plank	core	mat	time	3×20–60 s	W
side-plank	Side plank	core	mat	time	3×20–40 s/side	—
dead-bug	Dead bug	core	mat	reps	3×8–12/side	—
bird-dog	Bird dog	core	mat	reps	3×8–12/side	—
hollow-hold	Hollow hold	core	mat	time	3×15–30 s	—
pallof-press	Pallof press	core	resistance band	reps	3×10–12/side	—
suitcase-carry	Suitcase carry	core	kettlebell/dumbbell	time	3×30–45 s/side	AH
cat-cow	Cat–cow	mobility	mat	reps	1×10	—
worlds-greatest-stretch	World's greatest stretch	mobility	none	reps	1×5/side	DH
hip-flexor-stretch	Half-kneeling hip flexor stretch	mobility	mat	time	2×30 s/side	—
ninety-ninety	90/90 hip switches	mobility	mat	reps	2×8	—
thoracic-rotation	Thoracic rotation (side-lying)	mobility	mat	reps	2×8/side	—
hamstring-stretch	Supine hamstring stretch	mobility	mat	time	2×30 s/side	—
calf-stretch	Wall calf stretch	mobility	none	time	2×30 s/side	—

Slug	Name	Category	Equipment	Tracking	Default	Contraindication tags
seated-ankle-circles	Seated ankle circles	mobility	none	reps	1×10/direction	—
easy-walk	Easy walk	recovery	none	time	20–40 min	—
shadow-boxing	Shadow boxing intervals	conditioning	none	time	6×40 s	—
mountain-climber	Mountain climber	conditioning	mat	time	4×30 s	W
jumping-jack	Jumping jack	conditioning	none	time	4×30 s	I, J, AH
jump-rope	Jump rope	conditioning	jump rope	time	6×45 s	I, J, AH
burpee	Burpee	conditioning	none	reps	4×8	I, J, W
high-knees	High knees	conditioning	none	time	4×20 s	I, AH
bike-intervals	Exercise bike intervals	cardio	exercise bike	time	6×1 min	—
ball-mastery	Ball mastery (sole rolls, toe taps)	sport_skills (football, futsal)	ball	time	5×1 min	SLB, AH
wall-passing	Wall passing (both feet)	sport_skills (football, futsal)	ball, wall	time	4×2 min	SLB
cone-dribbling	Cone dribbling	sport_skills (football, futsal)	ball, cones	time	6×30 s	L, S, AH
acceleration-sprints	Acceleration sprints 10–20 m	sport_skills (football, futsal)	none	reps	6–8	I, S, AH
change-of-direction	Change-of-direction drill	sport_skills (football, futsal)	cones	reps	6–8	I, L, S, AH
single-leg-balance	Single-leg balance	mobility	none	time	3×30 s/side	SLB

Every system exercise MUST have instructions (3–6 steps), caution notes, and at least one alternative mapping in templates. Media URLs are left empty until licensed media is available (do not hotlink random videos).

25.3 Workout templates (seed)

Template	Intensity	Minutes	Exercises (sets × reps, rest)	Alternatives when restricted
Strength A — Upper & core	moderate	45	warm-up cat-cow + thoracic rotation; db-floor-press 3×12–15 (60 s); one-arm-db-row 3×10–15/side (60 s); push-up 3×AMRAP−2 (60 s); db-shoulder-press 3×10–15 (60 s); lateral-raise 2×15–20; dead-bug 3×10/side; plank 3×30–45 s	push-up → incline-push-up; db-shoulder-press → lateral-raise; plank → dead-bug
Strength B — Lower & core	moderate	45	goblet-squat 3×12–15; db-rdl 3×12–15; reverse-lunge 3×10/side; glute-bridge 3×15–20; standing-calf-raise 3×15–20; side-plank 3×30 s; bird-dog 3×10/side	goblet-squat → box-squat; reverse-lunge → single-leg-glute-bridge; standing-calf-raise → seated-calf-raise; step-up → hip-thrust
Strength B — ankle-safe (auto when ankle restricted)	moderate	40	box-squat 3×12–15; db-rdl 3×12–15; glute-bridge 3×15–20; single-leg-glute-bridge 3×10/side; seated-calf-raise 3×15 (only if pain-free); dead-bug; side-plank; hip-flexor-stretch	—
Full body 20 (short version)	moderate	20	goblet-squat or box-squat 2×15; push-up/incline 2×AMRAP−2; one-arm-db-row 2×12/side; glute-bridge 2×20; plank 2×30 s	as above
Low-impact conditioning	moderate	30	bike-intervals 6×1 min or shadow-boxing 6×40 s; kb-swing 4×15; mountain-climber 4×30 s; dead-bug	kb-swing → glute-bridge
Recovery	easy	30–45	easy-walk 20–30 min + cat-cow, hip-flexor-stretch, hamstring-stretch, calf-stretch, thoracic-rotation	calf-stretch skipped if ankle active

Template	Intensity	Minutes	Exercises (sets × reps, rest)	Alternatives when restricted
Mobility 15	easy	15	cat-cow, worlds-greatest-stretch, ninety-ninety, thoracic-rotation, hamstring-stretch	worlds-greatest-stretch → hip-flexor-stretch
Football & futsal skills (`futsal-skills`)	moderate	30	ball-mastery, wall-passing, cone-dribbling, acceleration-sprints	not scheduled while restricted
Per-sport packs (11)	moderate	30	4–6 sport drills each (31 drills, no contact or sparring): basketball-, handball-, volleyball-, racket-, hockey-, rugby-, swim-, combat-, run-, cycle- and athletic-skills	one weekly session uses the first sport's pack when the user plays a sport, has the sport focus, trains ≥ 3 days and is not restricted for that sport; every pack still works with no equipment

Every template includes a warm-up and a short cool-down. Strength templates use a 5-minute mobility warm-up (cat-cow, thoracic rotation, glute bridge).

Sport packs (all 12) use a RAMP-style dynamic warm-up of about 8–10 minutes: march or easy jog in place 2 min → leg swings 10/side → lateral lunge 6/side → walking knee hug 6/side → arm circles 45 s → 2–3 build-up runs to about 70% (swapped for marching when impact or sprinting is restricted). Swimming and cycling packs use a dryland version without running (march, arm circles, leg swings, cat-cow). Dosage rules, checked by tests: sprint, change-of-direction and approach drills rest ≥ 90 s between sets; jumping drills total ≤ 30 foot contacts per pack. Numbers, sources and open questions for the dietitian, physiotherapist and S&C coach are in `docs/expert-review-pack.md`.

25.4 MET reference (from the Adult Compendium of Physical Activities; verify values during implementation)

Activity	Condition	MET
Walking	< 3.2 km/h	2.0
Walking	3.2–4.4 km/h	3.0
Walking	4.4–5.2 km/h	3.5
Walking	5.2–6.0 km/h	4.3
Walking	≥ 6.0 km/h	5.0
Running	< 7.0 km/h	6.0
Running	7.0–8.7 km/h	8.3
Running	8.7–10.2 km/h	9.8
Running	10.2–11.7 km/h	11.0
Running	11.7–13.7 km/h	11.8
Running	13.7–15.3 km/h	12.8
Running	≥ 15.3 km/h	14.5
Cycling	< 16 km/h	4.0
Cycling	16–19.3 km/h	6.8
Cycling	19.3–22.5 km/h	8.0
Cycling	22.5–25.7 km/h	10.0
Cycling	≥ 25.7 km/h	12.0
Sports (games / practices)	per sport, from the sports catalogue	e.g. football/futsal 10.0 / 7.0; basketball 8.0 / 6.0; tennis 8.0 / 6.0; swimming 9.8 / 5.8
Sport drills (`sport_skills`)	—	7.0
Resistance training	moderate	3.5
Resistance training	vigorous	6.0
Circuit / kettlebell conditioning	vigorous	8.0
Jump rope	moderate	11.8
Mobility / stretching	—	2.3

Bracket boundaries are lower-inclusive. Workout sessions use the template intensity (moderate → 3.5, hard → 6.0, conditioning → 8.0, mobility → 2.3) unless an exercise-level MET exists.

25.5 Milestone catalogue

Key pattern	Title	Condition
weight_lost_1 / _3 / _5 / _10 / _15…	First 1 kg lost, 3 kg, 5 kg, then every 5 kg	7-day average ≤ start − N
weight_reached_{N}	{N} kg reached	7-day average ≤ N, for multiples of 5 between start and target
goal_target / goal_secondary	Target weight reached / Secondary target reached	7-day average ≤ target
workouts_1 / _5 / _10 / _25 / _50 / _100	First workout, 5, 10… workouts	Completed sessions count
strength_10 / _25	10 / 25 strength sessions	Category strength
walk_5k / run_5k / run_10k / cycle_10k / cycle_25k	First 5 km walk, etc.	Single activity distance ≥ threshold (good/fair GPS or manual)
training_hours_10 / _25 / _50 / _100	10 hours of training, etc.	Sum session + activity moving time
logging_streak_7 / _30	7-day logging streak	Consecutive local days with any log
weigh_in_30	30 days of weight tracking	30 distinct weigh-in days within 45 days
consistency_weeks_4 / _12	4 / 12 consistent weeks	Consecutive weeks with ≥ 80% of planned sessions completed

Streak copy is positive only ("Start a new streak today"), never "Streak lost".

25.6 Unit and display reference

kg ↔ lb (×2.20462262), cm ↔ in (÷2.54), km ↔ mi (÷1.609344), ml ↔ fl oz (÷29.5735). Pace display mm:ss per km or per mile; speed km/h or mph. Dates follow device locale; first day of week from profile.

26. Implementation phases

Each phase: (1) short plan, (2) implementation, (3) tests, (4) Phase Report. Do not start the next phase without approval.

Phase 1 — Foundation

Scope: Expo project (TypeScript strict, Expo Router in `src/app`, ESLint, Prettier, Jest), environment config, Supabase client + session persistence, auth screens (sign up, sign in, sign out, forgot/reset password via deep link, verify email), migrations for profiles, user_goals, user_preferences, user_equipment, consents, health_conditions, nutrition_targets, training_plans, recurring_events (+ updated_at trigger, new-user profile trigger, RLS), design system tokens + core components + dark/light theme, tab navigation shell with empty states, onboarding (all 9 steps, local draft, validation), domain calculations (BMR, TDEE, target, floor, macros, water, projection, units) with full unit tests, plan preview with calculation breakdown, profile editing screens for these fields, legal screens with placeholder-free disclaimer text, CI script.

Exit criteria: register → verify → onboard → see plan → kill app → reopen in tabs; profile editable; all calculation test vectors pass; pgTAP RLS tests pass for Phase 1 tables; typecheck and lint clean.

Phase 2 — Dashboard and progress

Scope: weight_entries, body_measurements, user_milestones migrations; weight logging (with outlier confirm, undo), weight chart (ranges, average, targets), stats, measurements + charts, goal progress and goal-reached flow, milestone engine (weight + logging milestones), dashboard v1 (greeting, weight card, targets card, quick actions, upcoming placeholder hidden until Phase 3), outbox + sync engine for weight and measurements, daily check-in.

Exit: weight flows work offline and sync without duplicates; chart correct against test data; milestones unlock once.

Phase 3 — Training

Scope: exercises, templates, scheduled workouts, sessions, sets, personal records (strength) migrations + seed; exercise library (search, filters, custom); injury filter; plan generator; calendar (week/month, move, skip, recurring edit); workout execution with drafts and rest timer; summary; history; sport event logging; progression suggestions; training milestones; dashboard "today's workout".

Exit: generated week for the seed profile matches Section 25.1; ankle restriction swaps exercises; recurring generation idempotent; workout survives app kill.

Phase 4 — Nutrition

Scope: foods (USDA import script + seed), portions, favourites, meal logs/items, saved meals, day status, water, nutrition targets display, TheMealDB provider + normaliser + measure parser + `recipes-proxy`, recipe browser/detail with servings, user recipe builder, meal planner, shopping list, nutrition rules for dashboard.

Exit: macros totals correct; snapshots immutable; TheMealDB recipes show "Nutrition unavailable"; scaling vectors pass; shopping list merges correctly; manual logging works with the API offline.

Phase 5 — GPS activity

Scope: activities migration + routes bucket; permission flows; location task; SQLite buffering; filter + metrics; live screen; pause/resume; recovery after kill; summary with map; manual activities; route deletion; activity PRs and milestones.

Exit: real-device walk and run of ≥ 2 km with screen locked on iOS and Android; distance within ~5% of a known route; pause excludes distance; killing the app mid-run recovers.

Phase 6 — Personal coach

Scope: recommendation engine + rules, recalibration + target_adjustments UI, training load, weekly report + history, notifications with reconciliation, remaining milestones.

Exit: every rule has unit tests (trigger and non-trigger); recalibration vectors pass; notifications respect caps and quiet hours.

Phase 7 — Production readiness

Scope: data export, account deletion function, privacy policy/terms content, accessibility audit, performance (startup < 2 s on mid-range device, list virtualisation), Maestro E2E flows, Sentry (optional), EAS production config, store listing (privacy nutrition labels / Data safety form: health, fitness, location, identifiers), real-device test matrix, submission.

Then V1.1: barcode + OFF, progress photos, monthly report, carb adjustment, GPX export, split PRs.

Phase 8 — Social (V1.2), in four reviewed stages

Each stage follows the same rules as earlier phases (plan, implementation, tests incl. pgTAP, Phase Report, approval before the next). Design: Section 33.

Stage	Scope	Acceptance
8A Friends and safety	Migrations `social_profiles`, `social_settings`, `friendships`, `blocks`, `reports`, `consents` gains `social`; RLS helpers (`are_friends`, `is_blocked`); handle validation and word filter (client + DB). Social setup sheet, App features switch, Friends screen (requests, list, handle search, invite link `pace://add/<handle>`), other-user profile, block / report, Settings → Social & privacy, privacy policy + terms update (legal version bump), export and deletion.	Two test accounts can find, request, accept, remove and block each other; blocked users are invisible to each other everywhere; nothing is visible without an opt-in; pgTAP proves isolation for owner / friend / stranger / blocked.
8B Feed, cheers, comments, push	`feed_items` with triggers on `workout_sessions`, `activities`, `user_milestones` and the snapshot whitelist; `cheers`; `comments`; `push_tokens`; Edge Function `social-push` (pg_net webhooks, batching, preferences, quiet hours, rate limits); Feed UI (paged), cheer and comment UI, notification settings, deep links. Security review before release.	A finished workout reaches a friend's feed only when shared, with no weight, route or clock time; deleting or abandoning the source removes it; ten cheers arrive as one push; quiet hours respected; no health detail in push payloads.
8C Clubs	`clubs`, `club_members`, invite codes (rotatable), roles owner / admin / member, limit 50; club home (Feed · Members · Board · Challenges), create / join / leave / remove / promote / delete; club audience `friends_and_clubs`; ownership transfer on account deletion.	Joining only by invite; non-members see nothing; the limit is enforced; owner removal and leaving work; deleting the owner's account transfers or removes the club.
8D Challenges and leaderboards	`challenges`, `challenge_participants`; server functions `leaderboard()` and `challenge_progress()` (aggregates only, daily caps, frozen results); challenge templates and UI; friends and club leaderboards (This week / This month; Sessions · Active minutes · Distance · Active days); `challenge_done` feed item and milestone.	Only opted-in users are ranked; caps apply (a 10 h manual activity counts as 4 h); results freeze at the end; injury-restricted users get no challenge nudges; no weight, body or food metric anywhere.

27. Acceptance criteria (V1)

1. A user can sign up, verify email, sign in, sign out and reset their password; invalid credentials show a clear error; sessions persist across restarts.
2. Onboarding can be completed, resumed after an app kill, and every field edited later in Profile.
3. Targets for the seed profile equal Section 25.1 exactly; the plan preview shows the breakdown.
4. Targets never fall below the floor; edits below the floor are blocked; every target change is versioned and visible.
5. Weight can be logged, edited, deleted (online and offline); chart and stats match the reference calculations; missing data shows as missing.
6. Measurements can be logged and charted.
7. Workouts can be planned, moved, skipped, completed and reviewed; history persists across reinstall (server).
8. Recurring sessions generate without duplicates over 8 weeks, including DST transitions.
9. Active/recovering injuries exclude contraindicated exercises in plans, templates and suggestions.
10. Meals can be logged from catalogue, custom foods, quick add, saved meals; totals are correct; historical items are unchanged after a food is edited.
11. Recipes can be searched and viewed with attribution; nutrition shows as unavailable unless calculated; servings scale only where quantities are parseable and a base is known.
12. A shopping list can be generated from planned meals and checked off offline.
13. Walk, run and cycle can be tracked, paused, resumed, finished and discarded; tracking continues with the screen locked when background permission is granted; routes are reviewable and deletable.
14. Milestones, PRs and weekly reports display correctly and never duplicate.
15. Daily recommendations appear with reasons and respect all safety rules.
16. User B can never read or modify user A's data (pgTAP + manual check).
17. Supported offline features work in airplane mode and sync without duplicates or loss when back online.
18. Account deletion removes all database rows and storage objects for the user; data export contains all user data.
19. All critical flows tested on at least one real iOS and one real Android device.
20. Typecheck, lint, unit tests, RLS tests and Maestro smoke flows pass in CI.

28. Edge-case catalogue

Each item: situation → required behaviour. (T) = must have an automated test.

28.1 Authentication and session

1 Sign up with an already registered email → show "Check your email to continue" (no account enumeration).

2 Sign in before verifying → explain and offer "Resend email" (rate-limited, 60 s cooldown).

3 Reset link expired or already used → error screen with "Request a new link".

4 Reset/verify deep link opened while app is closed (cold start) → handled after initialisation.

5 Token refresh fails while offline → keep using cached data, allow offline logging, re-authenticate when online; never lose the outbox.

6 Sign out with pending outbox items → warning "N changes not synced yet" with Sync now / Sign out anyway.

7 Different user signs in on the same device → previous user's local caches and outbox are not visible or synced (T).

8 Account deleted on another device → next request returns 401/user not found → sign out locally and clear data.

9 Weak password → zod rule: ≥ 8 characters with an uppercase letter, a lowercase letter and a number (Supabase `lower_upper_letters_digits`); a live checklist under the field ticks each rule off as it is met (sign-up, reset, change password). Signing in only checks a password was entered, so older passwords keep working.

28.2 Onboarding and profile

1 App killed mid-onboarding → resumes at last step with entered values.

2 Age < 18 → cannot continue; friendly explanation (T).

3 Imperial height 6 ft 2 in → 187.96 cm stored as 188.0; displays 6 ft 2 in (T).

4 Decimal comma input "108,5" → parsed as 108.5 (T).

5 Lose-weight goal with target ≥ current → validation error suggesting maintain (T).

6 Target implying BMI < 18.5 → blocked with explanation (T).

7 Weekly rate 1.2% → not selectable; 0.9% → warning confirmation (T).

8 No training days and no commitments → plan contains recovery walks and mobility suggestions only.

9 Commitments occupy all preferred days → scheduler places strength on other allowed days or reduces sessions; never violates MD-1 rule (T).

10 Max session 15–25 min → short templates used (T).

11 "No equipment" → bodyweight-only exercises (T).

12 Optional steps skipped → defaults shown with "default" badge.

13 Timezone changes (travel) → new events use the new timezone; existing `local_date` values unchanged.

14 Birthday passes → age changes only at next target recalculation (not silently).

15 User changes sex-for-calculation or height → recalculation proposal, not silent change.

28.3 Calculations

1 Safety floor raises target → `limited_by_floor = true` and message (T).

2 Deficit cap 25% TDEE applied for small TDEE (T).

3 Macros sum within ±10 kcal of target (T).

4 Carbs < 50 g after edits → warning (T).

5 Protein > 2.5 g/kg reference → warning.

6 User edits macros → calories derived; edits calories → carbs adjust (T).

7 Edit below floor → blocked (T).

8 Projection with zero or positive expected rate → no date shown (T).

9 Division by zero protection everywhere (distance 0, duration 0, servings 0) (T).

28.4 Weight and measurements

1 Multiple entries same day → daily value = mean; all raw entries listed (T).
2 Future date → blocked (T).
3 Value differs > 3 kg from 7-day average → confirmation (T).
4 Only 1–2 entries → no average line, "Log a few more" (T).
5 Gap > 7 days → average line breaks (T).
6 Unit switch → whole history converts in display; stored values unchanged.
7 Deleting an entry that unlocked a milestone → milestone stays.
8 Goal target reached → goal-reached flow, no further deficit without confirmation.
9 Weight gain trend → neutral copy, no projection date.
10 Editing an old entry offline and online on two devices → LWW by client_updated_at.

28.5 Training

1 Move workout onto MD-1 or making 3 hard days in a row → warning, allowed if confirmed.
2 Edit recurring event "this and future" → future planned occurrences regenerated; completed untouched (T).
3 Delete recurring series → future planned occurrences removed, history kept (T).
4 Rolling generation run twice → no duplicates (unique constraint + upsert) (T).
5 DST change week → event local time stays 18:00 (T).
6 App killed during workout → resume prompt with all sets.
7 Rest timer while app backgrounded → correct remaining time on return; local notification at end.
8 Session longer than 4 h → on finish ask to confirm/adjust end time.
9 Zero completed sets → status abandoned; not counted (T).
10 Bodyweight exercise → no weight input; volume excludes it (T).
11 Template edited after scheduling → scheduled snapshot unchanged (T).
12 Custom exercise deleted while in history → archived, history shows name (T).
13 Injury added after plan exists → future planned workouts re-filtered with "Adjusted" label (T).
14 Alternative also contraindicated → exercise removed, workout still valid; if < 3 exercises remain, suggest recovery session (T).
15 Two devices start the same scheduled workout → two sessions; scheduled workout marked completed once; history shows both.

28.6 GPS activity

1 Permission denied → manual entry offered; no crash.
2 Foreground only → banner "Keep PACE open"; tracking pauses when backgrounded is reported honestly in summary.
3 Approximate location → warning, quality poor (T for quality classification).
4 GPS acquiring > 60 s → user can start anyway; distance waits for accurate fixes.
5 Accuracy > 25 m points → rejected (T).
6 Speed spike (teleport) → rejected (T).
7 Stationary jitter → no distance added (T).
8 Pause → no distance across pause (T).
9 Signal loss 2 min then plausible jump → bridged; implausible → not counted (T).

10 Out-of-order or duplicate timestamps → rejected (T).

11 OS kills app → recovery prompt on relaunch; no duplicate activity (T for idempotent finish).

12 Phone reboot mid-activity → same recovery from SQLite.

13 Very short activity → save confirmation.

14 Activity across midnight → `local_date` = start date (T).

15 Changed activity type after save → metrics recomputed; filter caps of new type applied (T).

16 Upload fails → activity kept locally "Not synced"; retried.

17 Route deleted → summary kept, map hidden, storage object removed.

18 Low storage / SQLite write error → stop recording safely and warn; keep what was written.

28.7 Nutrition and recipes

1 Food missing macros → item allowed, "Partial data" badge; totals flag incomplete (T).

2 ml entered for a per-100 g food → not allowed unless a portion defines it.

3 Quantity 0, negative or > 5000 g → validation / confirmation (T).

4 Food edited/deleted after logging → logged snapshot unchanged (T).

5 Copy meal into a day that already has that meal type → items appended (T).

6 TheMealDB `meals: null` → empty state, not error (T).

7 TheMealDB unavailable → cached data or error with retry; manual logging works.

8 Recipe with no stated servings → base servings prompt before scaling (T).

9 Measures "1/2 tsp", "1 1/2 cups", "½", "2-3", "175g", "To taste", "" → parsed per 12.4 (T).

10 Scaling counts (3 eggs × 1/4) → 0.75 → "¾ (approx.)" (T).

11 Same TheMealDB recipe opened twice → one cached row (unique) (T).

12 Allergy keyword present in ingredient → recipe hidden from suggestions and flagged in browse.

13 Vegetarian user browsing beef recipe → flagged "Contains meat" (keyword list).

14 Shopping list: same ingredient different units → separate lines; same unit → summed; unparsed → raw line (T).

15 Water: undo within 5 s; > 10 L day → confirmation.

16 Logging after midnight for "yesterday's dinner" → date picker defaults to today but easy to change.

17 Day marked complete then items added → stays complete; user can untoggle.

28.8 Milestones, records, reports

1 Same milestone evaluated on two devices / retries → one row (unique) (T).

2 Bulk import → single summary notification (T).

3 Goal changed → locked weight milestones regenerated; unlocked kept (T).

4 Weight regained after milestone → not revoked, not re-awarded (T).

5 Activity with poor GPS → excluded from pace/distance PRs (T).

6 Source session deleted → PR recalculated (T).

7 Week with no weigh-ins → "No data" (T).

8 Average protein on 3 complete days of 7 → "based on 3 of 7 days" (T).

9 Current week report → labelled "In progress".

28.9 Coach and notifications

1 Rule conflicts (fatigue + workout reminder) → higher priority wins; training suggestion suppressed (T).

2 Dismissed card → hidden until tomorrow (T).

3 No data (new user) → onboarding tips only (T).

4 Injury active + match tomorrow → S3 shown, R1 still shown (rest), never "play" advice (T).

5 Notification permission denied → in-app only; settings explain how to enable.

6 More than 64 notifications needed → schedule nearest 14 days within limit (T).

7 Quiet hours → reminders moved to 07:00 or dropped if obsolete (T).

8 Timezone change → reconcile reschedules all.

28.10 Sync and data lifecycle

1 Create + delete before sync → nothing sent (T).

2 Parent/child order → child never synced before parent (T).

3 RLS/validation error → item marked failed, visible, not retried endlessly (T).

4 Network flapping → backoff, no duplicates (T).

5 Account deletion with pending outbox → confirmation, outbox cleared after success.

6 Deletion function partial failure → retry is idempotent (T in function tests).

7 Export of large history → streamed per table; works for 5 years of data.

29. Test plan and test vectors

29.1 Levels

Level	Tool	Scope
Unit	Jest	All `src/domain` functions (target ≥ 90% line coverage for domain), zod schemas, normalisers, parsers
Component	React Native Testing Library	Forms (validation messages), key cards, empty/error states
Integration	Jest + local Supabase	Repositories, sync engine against local DB
Database	pgTAP (`supabase test db`)	RLS isolation, constraints, triggers, unique indexes
E2E	Maestro	Sign up → onboarding → log weight → start/finish workout → log meal → sign out
Device	Manual checklist	GPS (screen locked, kill, reboot), notifications, offline, deletion

29.2 Calculation vectors

#	Input	Expected
C1	male, 108 kg, 187 cm, 23 y	BMR 2138.75
C2	female, 65 kg, 165 cm, 30 y	BMR 1370.25
C3	male, 80 kg, 180 cm, 40 y	BMR 1730.00
C4	C1 × moderate	TDEE 3315.0625
C5	C1, lose 0.5%/wk	target 2720, protein 162, fat 76, carbs 347
C6	C2 × light (1884.09), lose 0.5%/wk (desired 357.5)	target round_to_10(1526.6) = 1530; floor 1370 not limiting
C7	female, 50 kg, 160 cm, 35 y, sedentary, lose 1.0%/wk	BMR 1164; TDEE 1396.8; desired 550 → capped 349.2 (25% of TDEE); raw 1047.6 → 1050 → floor max(1200, 1160) = 1200 → target 1200, limited_by_floor = true
C8	C1, lose 1.0%/wk	desired 1188 → capped 750 → target round_to_10(2565.06) = 2570, projected 0.68 kg/wk
C9	weight 108 kg	water 3800 ml
C10	weight 40 kg	water 1500 ml (clamped)
C11	6 ft 2 in	187.96 cm; 108 kg = 238.1 lb
C12	DOB 2003-11-09, asOf 2026-11-08	age 22
C13	DOB 2000-02-29, asOf 2025-03-01	age 25

29.3 Weight vectors

#	Data	Expected
W1	7 days: 108.0, 107.6, 108.2, 107.4, 107.8, 107.2, 107.5	7-day average 107.67
W2	Same day 108.0 and 107.0	daily value 107.5
W3	Days 1, 2 only	no average
W4	21 daily values decreasing exactly 0.1 kg/day	slope −0.7 kg/week
W5	Recalibration: mean intake 2700, slope −0.2 kg/week, target rate 0.54, previous check also slow, adherence 0.9	observed TDEE = 2700 + (0.2/7)×7700 = 2920; propose −100 kcal
W6	Adherence 0.7	no proposal, adherence message
W7	Loss 1.2 kg/week at 100 kg	propose +150 kcal

29.4 Geo and activity vectors

#	Input	Expected
G1	(0, 0) → (0, 1)	111 195 m (±1 m)
G2	(0, 0) → (1, 0)	111 195 m (±1 m)
G3	5000 m in 1500 s	pace 300 s/km = 5:00 /km; 8:03 /mi; speed 12.0 km/h
G4	Run point with accuracy 40 m	rejected

#	Input	Expected
G5	Run points 100 m apart, 2 s apart (50 m/s)	second point rejected
G6	Pause between points A and B 500 m apart	0 m added
G7	Run 30 min at 10 km/h, 108 kg	MET 9.8 → 9.8 × 108 × 0.5 = 529 kcal
G8	Walk 30 min at 5 km/h, 108 kg	MET 3.5 → 189 kcal
G9	Activity started 23:50, ended 00:40 local	local_date = start day

29.5 Nutrition vectors

#	Input	Expected
N1	150 g of food with 165 kcal/31 g P/0 g C/3.6 g F per 100 g	247.5 kcal, 46.5 g P, 0 g C, 5.4 g F
N2	Measure "1 1/2 cups"	1.5 cup
N3	Measure "175g"	175 g
N4	Measure "2-3 cloves" ×2	4–6 cloves
N5	Measure "To taste" ×2	unchanged, to_taste
N6	400 g chicken, base 4 → 2 servings	200 g
N7	3 eggs, base 4 → 1 serving	0.75 → "¾ egg (approx.)"
N8	List: 200 g rice + 300 g rice + 1 cup rice	500 g rice; 1 cup rice (separate)
N9	User recipe total 2000 kcal, 4 servings	500 kcal/serving
N10	Workout volume 3 sets × 12 reps × 5 kg	180 kg

29.6 Device test matrix (minimum)

One recent iPhone (current iOS) and one older supported iPhone; one Pixel/Samsung on current Android and one mid-range Android with aggressive battery management (e.g. Xiaomi or Samsung). Test: screen-locked tracking 30 min, kill during tracking, airplane mode logging, notification delivery, dark/light, largest font size, VoiceOver/TalkBack on main flows.

30. Implementation rules for the coding agent

1. Inspect the repository before changes; preserve working code; explain any replacement.
2. Follow the phase order. Write a short plan before each phase.
3. Keep business logic in `src/domain`; UI never calculates targets, distances, totals or milestones.
4. Use typed, validated data everywhere; no `any`; generated database types.
5. Every schema change is a migration; never change the database manually.
6. Every screen implements loading, empty, error and success states.
7. Unit test every formula and every edge case marked (T). Report real test output.
8. RLS on every table with pgTAP tests.
9. Never hardcode the example user; dev seed only.
10. No fake functionality; hide or label incomplete features.
11. No invented nutrition; estimates labelled; GPS distance not presented as exact.
12. No training recommendations conflicting with recorded injuries.
13. No secrets in the client or repo.
14. Document setup, environment variables, migrations and deployment in README and `docs/`.
15. When a requirement in this document is unclear or impossible, stop and ask, or record the decision in `docs/decisions.md`.

31. Phase report template

```
# Phase N report
1. Summary of what was implemented
2. Created / modified files
3. Database migrations (file name + purpose)
4. Environment variables (new or changed)
5. Commands run and ACTUAL results: typecheck, lint, unit tests, RLS tests, E2E
6. Edge cases covered (IDs from Section 28) and any not covered
7. Known limitations and deviations from the spec (with reason)
8. Manual setup still required
9. Next phase and its plan
```

32. Kickoff prompt (Phase 1)

```
Read docs/PACE_SPEC.md completely. It is the source of truth.

Implement ONLY Phase 1 — Foundation (Section 26).

Before writing code:
1. Inspect the repository, package manager and existing dependencies.
2. Check for an existing Expo project and Supabase configuration.
3. Report conflicts or missing prerequisites (Node LTS, Supabase CLI, Docker, EAS CLI).
4. Write a short implementation plan for Phase 1 and list the files you will create.

Then implement Phase 1 exactly as specified, including:
- Expo + TypeScript strict + Expo Router (src/app), ESLint, Prettier, jest-expo.
- Environment handling (.env.example, app.config.ts fail-fast).
- Real Supabase Auth: sign up, verify email, sign in, sign out, password reset deep link, session persistence.
- Migrations + RLS + triggers for all Phase 1 tables (Section 20/21), pgTAP isolation tests.
- Design tokens, core UI components, dark and light themes, tab shell with empty states.
- 9-step onboarding with zod validation, local draft and resume.
- Domain calculations with all test vectors from Section 29.2.
- Plan preview with calculation breakdown and editable targets.
- Profile editing for Phase 1 fields; legal/disclaimer screens.
- README with installation, Supabase setup (local and hosted, EU region), launch instructions,
  development vs production configuration; docs/migrations.md.

Do not implement workout execution, nutrition logging, recipes, GPS or reports.
Do not fake authentication or backend success.

At completion run typecheck, lint, unit tests and RLS tests, then deliver the Phase Report (Section 31).
Do not start Phase 2 until Phase 1 is reviewed and approved.
```

33. Social (V1.2, Phase 8)

Friends, a feed of shared training, clubs, challenges and effort-only leaderboards. The goal is accountability and encouragement around training, never comparison of bodies. The principles in 3.4 are binding.

33.1 Concepts and defaults

- **Social is off until turned on** (Settings → App features → Social, like nutrition; plus a server kill switch `app_config.social_enabled`). Turning it on opens the setup sheet: (1) handle and display name, (2) what friends and clubs see, per type, **all off by default**, with the "never shared" list shown, (3) leaderboard opt-in; then the `social` consent is recorded with the policy version.
- **Audiences** per type (workouts, activities, milestones, challenge results): `private` · `friends` · `friends_and_clubs`. A single shared item can be changed or hidden afterwards.
- **Friends** are mutual (request → accept). Up to 500. Requests can be limited to nobody.
- **Clubs** are private and invite-only (link or code), up to 50 members, roles owner / admin / member.
- **Social writes are online-only** (not the offline outbox): permissions must be checked on the server and a queued request or cheer replayed later would be confusing. Reads are cached; offline shows a clear state.

33.2 Data model (all tables RLS-protected, pgTAP-tested)

Table	Key columns
`social_profiles`	`user_id` pk · `handle` citext unique (3–20, `[a-z0-9_.]`, reserved words and word filter) · `display_name` · `avatar_color` · `bio` ≤ 140 · `discoverable` · `banned_at`
`social_settings`	`user_id` pk · audience per type · `friend_requests_from` (everyone / nobody) · `in_leaderboards` · `share_volume` · notification types (cheers, comments, requests, clubs, challenges); quiet hours reuse the reminder settings
`friendships`	canonical pair `(user_low, user_high)` · `requested_by` · `status` pending / accepted · timestamps; declining deletes the row
`blocks`	`(blocker, blocked)`; a block removes the friendship and hides both users from each other everywhere
`feed_items`	`owner` · `kind` workout / activity / milestone / challenge_done / club_joined · `source_id` · `snapshot` jsonb (whitelist only) · `audience` · `local_date` · `created_at`
`cheers`	`(feed_item_id, user_id)` unique · `kind` ☐ ☐ ☐ ☐
`comments`	`feed_item_id` · `author` · `body` ≤ 280 (filtered) · `deleted_at`
`clubs`	`name` (filtered) · `description` · `avatar_color` · `owner` · `invite_code` · `member_limit` 50 · `week_start`
`club_members`	`(club_id, user_id)` · `role` · `joined_at`
`challenges`	`scope` friends / club · `club_id` · `creator` · `title` · `metric` sessions / active_minutes / distance_km / active_days · `mode` team_goal / personal_target / ranking · `target` · `starts_on` / `ends_on` (≤ 31 days) · `status`
`challenge_participants`	`(challenge_id, user_id)` · `joined_at` · `final_value` (frozen at the end)
`reports`	`reporter` · `target_type` profile / feed_item / comment / club · `target_id` · `reason` · `status`; service role only
`push_tokens`	`user_id` · Expo token · platform · `updated_at`; deleted on sign-out and account deletion

33.3 Access rules

- Security-definer helpers used by RLS: `are_friends(a, b)`, `is_blocked(a, b)`, `share_club(a, b)`, `can_view_feed_item(viewer, item)` (owner; accepted friend when the audience includes friends; club co-member when it includes clubs; never across a block; never for banned users).
- **Source tables keep owner-only RLS unchanged.** Other people only ever read `feed_items` snapshots and server aggregates.
- **Feed creation by trigger** on `workout_sessions`, `activities`, `user_milestones` (insert / update / delete): created only when that type's audience isn't private; skipped for abandoned sessions; updated on edit; deleted with the source.

- **Snapshot whitelist:** workout → title, category, duration, set count, volume (only with `share_volume`); activity → type, distance, moving time, average pace; milestone → title (weight milestones are never shared). Tested to contain no other keys.

33.4 Leaderboards and challenges

- Computed on the server (`leaderboard(scope, scope_id, metric, period)`, `challenge_progress(challenge_id)`) and returned as aggregates only.
- Metrics: sessions (completed or partial), active minutes, distance (walk / run / cycle), active days. **Never** weight, body, calories or food.
- **Daily caps** (anti-cheat and anti-overtraining): at most 240 active minutes, 60 km on foot and 150 km cycling counted per day. Manual activities count and are marked as manual.
- Periods: this week (club week start, or the user's for friends) and this month. Ties share a rank. Only users with `in_leaderboards` are ranked; others show as "not ranked". No notifications about rank changes.
- Challenge modes: **team goal** (the group's total reaches a target), **personal target** (each participant's own target), **ranking** (effort ranking among participants). Templates: "Move 150 minutes this week", "10 workouts in 30 days", "Club walks 200 km this month". Values freeze when the challenge ends; finishing creates a `challenge_done` feed item (if shared) and a milestone. Users with an active injury restriction (9.x) get no challenge nudges.

33.5 Screens

- **Tab bar stays at five tabs at most:** Home, Training, Social, Progress, and Nutrition or Profile. When nutrition and social are both on, Profile leaves the tab bar (it opens from the Home header avatar).
- **Social tab:** Feed · Friends · Clubs · Challenges. Feed cards (paged): avatar, name, summary ("Strength A · 45 min · 9 sets"), date, cheers (tap; long-press picks a kind), comments. Friends: requests, list, handle search (≥ 3 characters), Invite (share sheet with `pace://add/<handle>`). Other user's profile: their shared items, mutual clubs, ⋯ Remove / Block / Report.
- **Clubs:** create, join by link or code, club home Feed · Members · Board · Challenges; owners and admins rotate the invite, remove, promote, edit and delete; members leave.
- **Challenges:** templates, progress rings / bars, days left, participants, ranks in ranking mode.
- **Settings → Social & privacy:** audiences, `share_volume`, leaderboard opt-in, notification types, blocked users, Turn social off (hides the tab, deletes your feed items, removes you from leaderboards, keeps friends), Delete my social profile.
- **Safety menu on every person, item, comment and club:** Report (reasons), Block, Remove (own content). A report confirms "Thanks, we'll review within 24 h" and shows the support email.

33.6 Push notifications

- `expo-notifications` push token stored in `push_tokens` after permission (EAS `projectId` required).
- Edge Function `social-push`, called by database webhooks (pg_net) on friend requests / accepts, cheers, comments, club joins and challenge events. It batches ("Ana and 3 others cheered your run"), respects per-type settings and quiet hours, is rate-limited per recipient, sends through the Expo Push API (`EXPO_ACCESS_TOKEN` secret) and never puts health detail in a payload. Tapping opens the item (`pace://social/item/<id>`).

33.7 Moderation, privacy and accounts

- Word filter for handles, club names, bios and comments, shared between app and database (`src/domain/social/filter.ts`, synced by `scripts/sync-shared.mjs`), enforced by database triggers.
- Rate limits (`edge_rate_limits`): friend requests 30/day, comments 60/hour, club creation 5/day, handle search 60/hour.
- Reports queue reviewed in the Supabase dashboard per the runbook in `docs/release/operations.md`; bans set `social_profiles.banned_at` (hidden everywhere, social writes refused).

- Privacy policy and terms gain a social section (what others can see, community rules, reporting); legal version bump; store forms updated for user-generated content and other users seeing activity.
- Export includes the user's own social rows. Account deletion cascades social data; clubs the user owns pass to the longest-standing admin, then member; empty clubs are deleted.

33.8 Testing

- pgTAP per table group (`supabase/tests/database/93_social_*.test.sql`): visibility for owner / friend / stranger / blocked / club member / banned; feed triggers; snapshot whitelist; caps and freezing; ownership transfer.
- Unit (`src/domain/social`): handle rules and filter, audience resolution, snapshot builder, ranking (ties, opt-out), challenge progress (caps, windows, time zones), push batching text.
- Component: setup sheet defaults off, feed card shows no forbidden fields, cheer toggle, safety menus.
- E2E: web Playwright with a mocked two-user store; Maestro on a device with two test accounts (the second driven through Supabase as that user). Security review of RLS helpers and `social-push` before 8B ships.

Appendix A — Glossary

BMR: basal metabolic rate. TDEE: total daily energy expenditure. MET: metabolic equivalent of task. RPE: rate of perceived exertion (0–10). MD-1 / MD+1: day before / after a match. RLS: Row Level Security. LWW: last write wins. Outbox: local queue of pending writes. Snapshot: copy of nutrition values stored with a log entry.

Appendix B — External references

- Mifflin MD, St Jeor ST et al. (1990). A new predictive equation for resting energy expenditure. Am J Clin Nutr.
- Adult Compendium of Physical Activities (2024 update) — MET values.
- International Society of Sports Nutrition position stand: protein and exercise (2017) — 1.4–2.0 g/kg guidance.
- Foster C. et al. (2001). A new approach to monitoring exercise training (session RPE).
- TheMealDB API: https://www.themealdb.com/api.php
- Open Food Facts API: https://openfoodfacts.github.io/documentation/
- USDA FoodData Central: https://fdc.nal.usda.gov/
- Supabase docs: Auth with React Native, Row Level Security, Storage access control, Edge Functions.
- Expo docs: expo-location (background location), expo-task-manager, expo-notifications, EAS Build.